
Transformadors

PID_00290595

Jordi de la Torre Gallart

Temps mínim de dedicació recomanat: 5 hores



Jordi de la Torre Gallart

L'encàrrec i la creació d'aquest recurs d'aprenentatge UOC han estat coordinats pel professor: Jordi Casas Roma

Com citar aquest recurs d'aprenentatge amb l'estil Harvard:

De la Torre Gallart, J. (2024). *Transformadors*. [Recurs d'aprenentatge textual]. 1a. ed. Barcelona: Fundació Universitat Oberta de Catalunya (FUOC).

Primera edició: febrer 2024

© d'aquesta edició, Fundació Universitat Oberta de Catalunya (FUOC)

Av. Tibidabo, 39-43, 08035 Barcelona

Autoria: Jordi de la Torre Gallart

Producció: FUOC

Tots els drets reservats

Cap part d'aquesta publicació, incloent-hi el disseny general i la coberta, no pot ser copiada, reproduïda, emmagatzemada o transmesa de cap manera ni per cap mitjà, tant si és elèctric com mecànic, òptic, de gravació, de fotocòpia o per altres mètodes, sense l'autorització prèvia per escrit del titular dels drets.

Índex

Introducció	5
Objectius	6
1. Introducció als transformadors	7
1.1. Evolució històrica dels mecanismes d'atenció	8
1.2. Desafiaments associats a l'arquitectura de les xarxes recurrents	9
2. Arquitectura dels transformadors	11
2.1. El mecanisme d'autoatenció	11
2.1.1. Definició formal	11
2.1.2. Definició conceptual: una analogia amb les bases de dades	14
2.1.3. Autoatenció múltiple	15
2.2. Transformadors	16
2.2.1. Notació	17
2.2.2. Elements constitutius del transformador	18
2.3. Tipus de transformadors	23
2.3.1. Transformadors seqüència a seqüència	23
2.3.2. Transformadors bidireccionals o d'autocodificació	26
2.3.3. Transformadors autoregressius	29
2.4. Transformadors per a processament del llenguatge natural (NLP)	33
2.4.1. Desenvolupaments previs: <i>embeddings</i>	33
2.4.2. Inicis dels transformadors per a NLP	34
2.5. Transformadors per a Visió (ViT)	35
2.5.1. Classificació	36
2.5.2. Detecció	38
2.5.3. Segmentació	41
2.6. Transformadors per a àudio	43
2.6.1. Senyals d'àudio, tractament i <i>tokenització</i>	43
2.6.2. Exemples de transformadors d'àudio	44
2.7. Transformadors multimodals	45
2.7.1. Entrades multimodals	46
2.7.2. Variants d'autoatenció en un context multimodal	46
2.7.3. Arquitectures multimodals	50

2.8. Aplicacions dels transformadors multimodals.....	50
Resum	52
Glossari	53
Notació	54

Introducció

Els **transformadors** són una arquitectura de xarxa neuronal dissenyada originalment per a la transformació de dades seqüència a seqüència, com a alternativa a les xarxes neuronals recurrents. El seu àmbit d'aplicació actual és més general. S'ha convertit en una arquitectura generalista capaç de competir i fins i tot superar una altra mena de xarxes especialitzades com les recurrents i les convolucionals. El seu ús actual s'estén a tractament de text, so, imatge, aprenentatge per reforç, entre altres, així com a aplicacions amb dades d'entrada heterogènies que utilitzen diferents classes de dades d'entrada.

La majoria de les tècniques que estudiarem en aquest mòdul han estat desenvolupades en els últims anys i, per tant, estan sotmeses a evolució. L'objectiu principal d'aquest mòdul és proveir el lector del context necessari perquè sigui capaç de comprendre els articles de recerca més recents relacionats amb el tema. Els diferents apartats se centren a introduir els fonaments matemàtics i algorítmics de tots els elements integrants de les xarxes de transformadors, per a així comprendre el funcionament d'aquesta arquitectura.

Proposades inicialment a Vaswani i altres (2017), des de la seva introducció es van convertir en l'estat de l'art per al tractament dels problemes típics de processament del llenguatge natural (NLP). Posteriorment, la seva aplicació s'ha anat ampliant a altres àmbits, i s'ha convertit en una eina principal per resoldre problemes més enllà del NLP.

El mòdul fonamental i distintiu de l'arquitectura és un sistema d'**autoatenció**, això és, la denominada atenció a la mateixa seqüència. Aquest mecanisme d'autoatenció deriva del sistema d'**atenció** introduït en els anys anteriors com a mitjà per intentar resoldre alguns problemes inherents al processament de llargues seqüències.

En aquest mòdul didàctic introduïrem l'arquitectura dels transformadors (*transformers*). Iniciarem el mòdul fent una mirada històrica a la casuística que va portar a la introducció i posterior evolució dels mecanismes d'atenció fins a arribar a una solució estable basada en els transformadors.

A continuació estudiarem els diferents components que conformen aquesta arquitectura, així com les diferents variacions que poden existir.

Finalment, veurem algunes de les aplicacions per a aquests models, començant per les basades en processament del llenguatge i per a la classificació d'imatges.

Objectius

En els materials didàctics d'aquest mòdul trobarem les eines indispensables per assimilar els objectius següents:

- 1.** Conèixer la casuística que va portar a la necessitat de plantejar una arquitectura alternativa per tractar les tasques de transformació de **seqüència a seqüència**.
- 2.** Conèixer els mòduls típics que integren l'arquitectura interna dels models basats en els transformadors, així com les diferents variacions.
- 3.** Conèixer com aplicar aquests models per a la solució de problemes de processament de llenguatge natural.
- 4.** Conèixer com aplicar aquests models per a la solució de problemes de processament d'imatge.
- 5.** Conèixer les possibilitats d'aquests models per a la resolució de problemes més complexos que utilitzin entrades de diferent naturalesa.

1. Introducció als transformadors

Les xarxes neuronals completament connectades són aproximadors universals. Ja el 1989, Hornik i altres van demostrar que a partir d'un conjunt definit de nodes d'entrada, amb únicament una capa oculta i amb suficients nodes, és possible aproximar qualsevol funció contínua amb la precisió desitjada (Hornik i altres, 1989).

Malgrat aquesta realitat teòrica matemàtica, la implementació pràctica és més complexa i fa que sigui difícil trobar el nombre de nodes i valor dels pesos adequats per aconseguir l'aproximació desitjada.

De cara a facilitar el procés d'optimització de les xarxes, històricament, s'han utilitzat priors estructurals que permeten restringir l'espai d'optimització a un subconjunt més acotat als valors esperats. Així, en el cas d'imatge, per exemple, les xarxes convolucionals exploten en el seu benefici predictiu la propietat inherent del nostre entorn que les regularitats estadístiques útils per realitzar una predicció solen estar situades en un entorn de proximitat espacial. D'altra banda, les xarxes recurrents, optimitzades per al tractament de dades seqüencials, exploten la propietat de localitat temporal, això és, que en certa manera la informació més rellevant per predir un esdeveniment sol estar en l'entorn més pròxim. Aquestes arquitectures, per tant, estan optimitzades per treure partit de les regularitats espacials i temporals presents en els senyals sensorials d'interès predictiu típics del nostre món (Ioannou, 2018).

Malgrat que aquests priors estructurals solen facilitar el procés d'aprenentatge, en alguns casos poden suposar un desavantatge. Les xarxes neuronals recurrents es troben sovint amb problemes relacionats amb la resolució de dependències de llarg abast, que les arquitectures existents no són capaces de resoldre. Limitacions en la memòria interna disponible, així com dificultats amb la propagació de gradients a través d'una llarga seqüència, poden ser causa d'un rendiment insatisfactori en les funcions encomanades.

Els models basats en mecanismes d'atenció neixen precisament amb l'objectiu d'abordar aquestes dependències mal resoltes, com un mecanisme que pretén modelitzar matemàticament l'emfatització dels atributs rellevants a l'hora de fer una predicció.

1.1. Evolució històrica dels mecanismes d'atenció

L'objectiu d'aquest subapartat no és fer una presentació exhaustiva de totes les modalitats de mecanismes d'atenció existents, però sí introduir el concepte i, posteriorment, presentar l'evolució que va donar lloc al sistema d'**autoatenció** utilitzat en els transformadors.

La importància de l'arquitectura dels transformadors es pot equiparar amb la que va suposar en el seu moment la proposta d'altres invencions anteriors esmentades en aquest mòdul, com les xarxes convolucionals i les xarxes recurrents LSTM. Com ja hem dit amb anterioritat, suposa una millora substancial respecte a les seves predecessores, convertint-se en una arquitectura generalista que permet no únicament tractar sinó també integrar en la predicció de manera elegant dades de naturalesa diversa.

La fita més important de la classe de xarxes neuronals basades en mecanismes d'atenció ha estat l'eliminació de la necessitat de la recurrència per al tractament de dades seqüencials, substituint la representació de les dependències d'un estat intern de la màquina a un mètode d'atendre en paral·lel les parts rellevants del conjunt de la seqüència. Bàsicament, convertir la dependència temporal en una d'espacial, codificant la temporalitat com una part addicional de l'entrada.

L'evolució històrica de les idees que van portar a l'estat de l'art actual pel que fa a atenció, comença al voltant de 2014. L'interès per dotar d'elements explicatius els models predictius, va portar al disseny de sistemes basats en atenció com a forma d'identificar els objectes presents en imatges que tenien més importància per a la realització de les prediccions de la xarxa neuronal. El 2015, es proposa la utilització d'aquests mecanismes en una aplicació de processament del llenguatge natural (NLP), concretament per a la traducció de text (Bahdanau i altres, 2014). Fins a aquest moment s'havien utilitzat arquitectures codificador-descodificador en les quals la primera part de la xarxa s'encarregava de codificar en un vector intern la frase d'entrada en l'idioma original i a continuació el descodificador la traduïa al llenguatge objectiu. Els autors d'aquesta nova proposta van conjecturar i van demostrar experimentalment que la utilització d'informació dels estats interns del codificador, i no únicament del vector intern final, millorava els resultats de rendiment.

A partir d'aquesta primera proposta es genera un nou corrent de recerca en aquest sentit, que fa que el 2017 es presenti un article que va canviar definitivament el paradigma de disseny cap a una nova arquitectura. A l'article «Attention is all you need» (Vaswani i altres, 2017), es presenta una proposta radical en la qual s'elimina totalment la recurrència, per dissenyar una xarxa formada per la combinació de mòduls d'**autoatenció**, normalització i capes completament connectades, denominada pels autors xarxa de **transformadors**, aconseguint resultats al nivell de l'estat de l'art i eliminant les complexitats associades a l'entrenament de les xarxes neuronals recurrents.

1.2. Desafiaments associats a l'arquitectura de les xarxes recurrents

Les xarxes recurrents van sorgir com una manera d'abordar el tractament de les dades seqüencials. Aquest tipus d'arquitectura s'adapta a les característiques d'aquestes dades, permetent l'entrada en l'ordre establert per la seqüència. Conceptualment, la xarxa és capaç de mantenir un estat intern que recull les dependències respecte a l'entrada. Aquestes poden ser de curt, mitjà o llarg abast. Idealment, la xarxa hauria de ser capaç de recordar i establir les relacions entre aquestes dependències de diferent naturalesa per predir sortides futures.

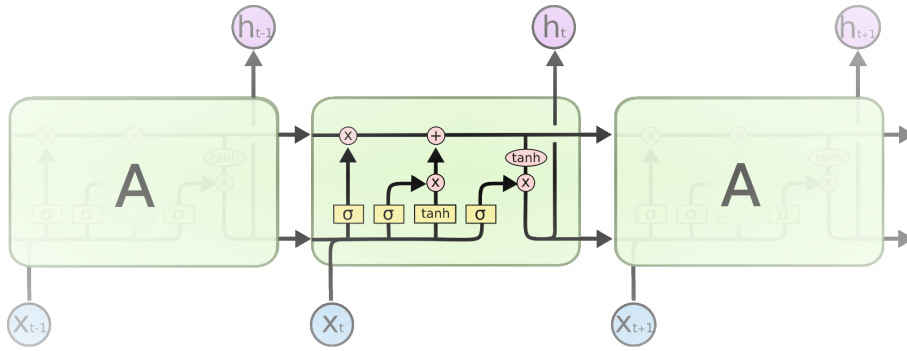
Aquest tipus de xarxes s'utilitza com a mòdul dins de models per abordar problemes dels següents tipus:

- 1) **Models vector-seqüència.** A partir d'un vector de mida fixa d'entrada es genera una seqüència de dimensió arbitrària. Un exemple seria l'etiquetatge d'imatges, on l'entrada és una imatge i la sortida un text explicatiu.
- 2) **Models seqüència-vector.** L'entrada és un vector de dimensió arbitrària i la sortida un de dimensió fixa. Un exemple seria l'anàlisi de sentiment, on a partir d'una seqüència d'entrada de dimensió variable es genera un valor binari que indica si el sentiment d'aquesta seqüència és positiu o negatiu.
- 3) **Models seqüència-seqüència.** Tant l'entrada com la sortida són seqüències de dimensió variable. Un exemple seria la traducció d'un text d'un idioma a un altre de diferent.

L'experiència pràctica demostra que les arquitectures basades en xarxes neuronals recurrents dissenyades per abordar aquests problemes no únicament requereixen més èpoques d'entrenament que les xarxes convolucionals, sinó que presenten una sèrie d'inconvenients relacionats amb l'alta profunditat de les xarxes desplegades, com la dificultat per resoldre les dependències de llarg abast, explosió i esvaïment de gradients i dificultat per a la programació en paral·lel de la computació necessària per dur a terme l'optimització.

Si bé arquitectures com les LSTM (Hochreiter y Schmidhuber, 1997) van suposar un avanç molt significatiu per a la incorporació de dependències de llarg abast, aquestes continuen presentant alguns problemes amb les dependències de molt llarg abast.

Figura 1. Diagrama bàsic representatiu d'una xarxa recurrent amb cel·les LSTM



El desplegament de les xarxes que cal realitzar durant l'entrenament sol incrementar les probabilitats d'explosió i esvaïment de gradients, amb els subseqüents efectes sobre l'optimització.

Quan les seqüències d'entrada són llargues, és necessari desplegar la xarxa en tota la seva extensió per calcular els gradients associats al tractament de la cadena. Això incideix no únicament en el punt anterior (gradients), sinó també en les necessitats de recursos computacionals requerits per al seu tractament (temps i espai).

2. Arquitectura dels transformadors

En aquest apartat atendrem els detalls que conformen l'arquitectura dels transformadors.

Els transformadors neixen amb l'objectiu de superar els desafiaments propis de les xarxes recurrents enunciats a l'apartat anterior. Aquesta nova arquitectura elimina la recurrència. Tots els elements de la seqüència es tracten al mateix temps. Els mecanismes d'atenció permeten la connexió directa d'elements situats a molta distància. Els problemes associats als gradients també es minimitzen, perquè tota la seqüència es calcula al temps i, per tant, els problemes de gradient passen a dependre únicament de la profunditat de la xarxa. D'altra banda, sabem que els problemes relacionats amb la profunditat de la xarxa van quedar resolts anys enrere amb l'adveniment de les xarxes residuals (He i altres, 2016). Els transformadors són també xarxes residuals i, per tant, resolen també el problema de profunditat de la xarxa utilitzant els mateixos mecanismes introduïts a He i altres (2016).

Finalment, l'entrenament a la pràctica demostra requerir menys èpoques i, per a cadascuna d'elles, el tractament en paral·lel de la computació és molt més senzill d'implementar.

2.1. El mecanisme d'autoatenció

El mecanisme d'autoatenció és la part principal i distintiva de l'arquitectura dels transformadors. Comprendre'n el funcionament és clau per endinsar-se en les especificitats de l'arquitectura.

2.1.1. Definició formal

Sigui $\mathbf{X}$ una seqüència d'entrada formada per $\ell_{\mathcal{X}}$ elements, cadascun d'ells de dimensió d_e , $\mathbf{X} \in \mathbb{R}^{d_e \times \ell_{\mathcal{X}}}$. Sigui $\mathbf{Z}$ una seqüència denominada de context formada per $\ell_{\mathcal{Z}}$ elements, cadascun d'ells de dimensió d_e , $\mathbf{Z} \in \mathbb{R}^{d_e \times \ell_{\mathcal{Z}}}$.

Siguin $(\mathbf{W}_{\mathbf{q}} \in \mathbb{R}^{d_{\text{attn}} \times d_e}, \mathbf{b}_{\mathbf{q}} \in \mathbb{R}^{d_{\text{attn}}}), (\mathbf{W}_{\mathbf{k}} \in \mathbb{R}^{d_{\text{attn}} \times d_e}, \mathbf{b}_{\mathbf{k}} \in \mathbb{R}^{d_{\text{attn}}})$ i $(\mathbf{W}_{\mathbf{v}} \in \mathbb{R}^{d_{\text{out}} \times d_e}, \mathbf{b}_{\mathbf{v}} \in \mathbb{R}^{d_{\text{out}}})$ tres parells de matriu-vector representatius de tres transformacions lineals aplicables a la seqüència d'entrada $\mathbf{X}$ i a la seqüència de context $\mathbf{Z}$.

Siguin $\mathbf{Q}$, $\mathbf{K}$ i $\mathbf{V}$ les matrius obtingudes després d'aplicar les transformacions lineals referides, on $\mathbf{Q} = \mathbf{W}_q \mathbf{X} + \mathbf{b}_q \mathbf{1}^T \in \mathbb{R}^{d_{attn} \times \ell_{\mathcal{X}}}$, $\mathbf{K} = \mathbf{W}_k \mathbf{Z} + \mathbf{b}_k \mathbf{1}^T \in \mathbb{R}^{d_{attn} \times \ell_{\mathcal{Z}}}$ i $\mathbf{V} = \mathbf{W}_v \mathbf{Z} + \mathbf{b}_v \mathbf{1}^T \in \mathbb{R}^{d_{out} \times \ell_{\mathcal{Z}}}$.

Sigui $\mathbf{S}$ la funció de **similitud**, definida en aquest cas com $\mathbf{S} = \frac{1}{\sqrt{d_{attn}}} \mathbf{K}^T \mathbf{Q} \in \mathbb{R}^{\ell_{\mathcal{Z}} \times \ell_{\mathcal{X}}}$.

Llavors, l'equació d'atenció de la seqüència d'entrada $\mathbf{X}$ sobre la seqüència de context $\mathbf{Z}$ es pot expressar com:

$$\tilde{\mathbf{V}} = \mathbf{V} \cdot \text{softmax}(\mathbf{S}) \quad \text{on} \quad \tilde{\mathbf{V}} \in \mathbb{R}^{d_{out} \times \ell_{\mathcal{X}}} \quad (1)$$

La funció *softmax* refereix a l'aplicació de la funció de normalització *softmax* per cada fila de la matriu. Per tant, cada element de la matriu depèn de tots els altres elements pertanyents a la mateixa fila.

Quan la seqüència d'entrada i el context coincideixen, això és $\mathbf{X} = \mathbf{Z}$, llavors parlem d'atenció a la mateixa seqüència o, més breument, autoatenció.

L'algorisme 1 mostra en versió algorítmica com es calcula l'autoatenció.

Algorisme 1 Esquema bàsic d'atenció amb una única consulta (versió seqüencial)

Entrada: $e \in \mathbb{R}^{d_{in}}$, la representació *token* d'entrada actual

Entrada: $e_t \in \mathbb{R}^{d_{in}}$, representació vectorial dels *tokens* de context $t \in [T]$

Sortida: $\tilde{v} \in \mathbb{R}^{d_{out}}$, representació vectorial de la combinació de *token* i context.

Paràmetres: $W_q, W_k \in \mathbb{R}^{d_{attn} \times d_{in}}, b_q, b_k \in \mathbb{R}^{d_{attn}}$, paràmetres de projecció lineal de consulta i clau.

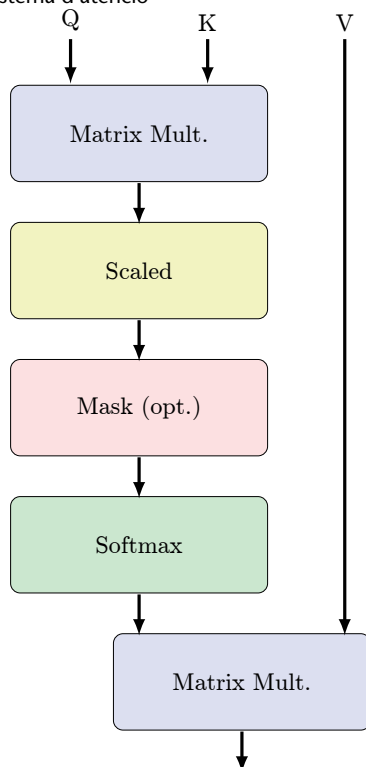
Paràmetres: $W_v \in \mathbb{R}^{d_{out} \times d_{in}}, b_v \in \mathbb{R}^{d_{out}}$ paràmetres de la projecció lineal de valor

- 1) $q \leftarrow W_q e + b_q$
 - 2) $\forall t : k_t \leftarrow W_k e_t + b_k$
 - 3) $\forall t : v_t \leftarrow W_v e_t + b_v$
 - 4) $\forall t : \alpha_t \leftarrow \frac{\exp(q^T k_t / \sqrt{d_{attn}})}{\sum_u \exp(q^T k_u / \sqrt{d_{attn}})}$
 - 5) **return** $\tilde{v} = \sum_{t=1}^T \alpha_t v_t$
-

Com s'indica a la versió presentada en l'algorisme 1, aquest càlcul es realitza per a tots els elements de la seqüència. Amb l'objectiu d'optimitzar en temps la computació, és d'interès paral·lelitzar la computació en la mesura que si-

gui possible. Una manera de fer-ho és mitjançant la configuració matricial de l'algorisme. A la figura 2 es mostra mitjançant diagrama de blocs les operacions matricials necessàries per dur a terme el procés. El mòdul *Màscara*, que es mostra al diagrama i es marca com a opcional, és una operació que, si calgués, permet anul·lar l'efecte de part de les seqüències. L'algorisme 2 és la versió matricial alternativa a l'algorisme 1, que permet explotar les possibilitats de càlcul en paral·lel i, per tant, accelerar la computació.

Figura 2. Diagrama de blocs representatiu de les operacions matricials involucrades en els càlculs relacionats amb el sistema d'atenció



L'operació d'emascarat permet el disseny d'estratègies d'entrenament alternatives per resoldre problemes diferents utilitzant la mateixa arquitectura. El sistema d'**autoatenció sense màscara** tracta tota la seqüència com a context $Z = X$, ($Mask \equiv 1$), donant lloc a transformadors amb contextos bidireccionals. Aplicant màscares als elements de la seqüència situats a un dels costats, tenim els transformadors de context unidireccional, mentre que en aquells en els quals tenim dues seqüències d'entrada, és possible aplicar la seqüència creuada emascarant els elements d'una de les seqüències d'entrada utilitzant com a context l'altra.

Algorisme 2 $\hat{V} \leftarrow \text{atencio}(X, Z | W_{qkv}, \text{Mask})$ (versió matricial)

/* Calcula un (únic) capçal d'atenció amb màscara */

Entrada: $X \in \mathbb{R}^{d_e \times \ell_x}$ i $Z \in \mathbb{R}^{d_e \times \ell_z}$, representacions vectorials de les seqüències principal i de context**Sortida:** $\hat{V} \in \mathbb{R}^{d_{out} \times \ell_x}$, representació vectorial actualitzada de X , recollint la informació de la seqüència Z **Paràmetres:** W_{qkv} , consistent en: $W_q \in \mathbb{R}^{d_{attn} \times d_e}$, $b_q \in \mathbb{R}^{d_{attn}}$; $W_k \in \mathbb{R}^{d_{attn} \times d_e}$, $b_k \in \mathbb{R}^{d_{attn}}$; $W_v \in \mathbb{R}^{d_{out} \times d_e}$, $b_v \in \mathbb{R}^{d_{out}}$.**Hiperparàmetres:** $\text{Mask} \in \{0,1\}^{\ell_z \times \ell_x}$

- 1) $Q \leftarrow W_q X + b_q 1^T$ [[Query $\in \mathbb{R}^{d_{attn} \times \ell_x}$]]
- 2) $K \leftarrow W_k Z + b_k 1^T$ [[Key $\in \mathbb{R}^{d_{attn} \times \ell_z}$]]
- 3) $V \leftarrow W_v Z + b_v 1^T$ [[Value $\in \mathbb{R}^{d_{out} \times \ell_z}$]]
- 4) $S \leftarrow K^T Q$ [[Score $\in \mathbb{R}^{\ell_z \times \ell_x}$]]
- 5) $\forall t_z, t_x$, if $\neg \text{Mask}[t_z, t_x]$ then $S[t_z, t_x] \leftarrow -\infty$
- 6) **return** $\hat{V} = V \cdot \text{softmax}(S / \sqrt{d_{attn}})$

Informalment, el transformador, a partir d'una sèrie d'atributs de representació de la seqüència d'entrada, fa una mitjana ponderada entre els mateixos basant-se en un càlcul de similitud. Els pesos aplicats a cadascun d'aquests atributs és proporcional al resultat de calcular la similitud entre parells d'atributs d'entrada. Aquesta comparació es repeteix diverses vegades amb les dades precedents de l'anterior capa. La primera capa efectua comparacions entre parells d'atributs, la següent entre parells de parells d'atributs, i així successivament. Segons s'avança en la profunditat de la xarxa, el nombre d'atributs combinats augmenta exponencialment, permetent obtenir en les últimes capes diferents formes d'acció combinada de tots els atributs de la seqüència.

2.1.2. Definició conceptual: una analogia amb les bases de dades

Els noms originals utilitzats per definir les transformacions lineals utilitzades en el càlcul de l'autoatenció fan referència a una analogia conceptual amb les consultes pròpies de les bases de dades relacionals.

En una base de dades tenim consultes (*queries*, Q), claus (*keys*, K) i valors (*values*, V). En realitzar una consulta Q sobre un conjunt de claus $K_1, K_2, \dots, K_N$, la base de dades reporta com a resultat una sèrie de valors $V_1, V_2, \dots, V_N$.

El mecanisme d'atenció dels transformadors ve a ser una versió probabilística d'aquest procés. Una funció de similitud compara la consulta Q amb cadas-

Autoatenció

Query, *key* i *value* són les tres transformacions lineals de les dades d'entrada que constitueixen la base del sistema d'autoatenció. El concepte d'autoatenció es deriva precisament del fet que les consultes Q es deriven de la naturalesa de la mateixa seqüència. El parell Q-K permet establir similituds entre «el buscat» (Q) i «l'oposat» (K). El valor de sortida serà una combinació lineal dels valors (V) sobre la base de la similitud entre cada parell Q-K.

cuna de les claus K . El resultat és un vector que pot ser interpretat com la similitud de la consulta Q amb cadascuna de les claus K . Aquest valor serveix posteriorment per calcular un pes que s'utilitza a continuació per calcular el valor final, com una combinació lineal dels valors d'entrada. La funció de similitud pot ser definida de diferents maneres (vegeu l'equació 2). És una decisió de disseny triar l'una o l'altra. Com hem observat en les equacions i algorismes presentats al subapartat anterior, els transformadors que estudiarem utilitzen com a funció de similitud el producte escalar escalat.

$$s_i = f(Q, K_i) = \begin{cases} Q^T K_i & \text{producte escalar} \\ \frac{Q^T K_i}{d} & \text{producte escalar escalat} \\ Q^T W K_i & \text{producte escalar general} \\ W_Q^T + W_K^T K_i & \text{similitud additiva} \\ \text{altres} & \text{kernels, etc.} \end{cases} \quad (2)$$

Calculada la similitud, obtenim per a una consulta concreta Q un valor per clau, és a dir, un vector de dimensió igual al nombre de claus. Valors alts de similitud indiquen un alt grau de coincidència. Per tant, com la intenció és crear una funció de cerca probabilística, s'utilitza aquest valor per calcular la probabilitat de coincidència aplicant una funció *softmax* (vegeu l'equació 3), i obtenint com a resultat una probabilitat del valor de la posició i per a una parella consulta-clau determinada.

$$\omega_i = \frac{\exp(s_i)}{\sum_j \exp(s_j)} \quad (3)$$

Finalment, s'obindrà un conjunt de valors com a combinació lineal dels valors font i el pes de cadascun d'ells en la comparació entre consulta-clau (vegeu l'equació 4).

$$A = \sum \omega_i V_i \quad (4)$$

2.1.3. Autoatenció múltiple

El mecanisme que hem descrit als subapartats anteriors permet realitzar una transformació utilitzant el superconjunt (Q, K, V) des de l'espai inicial $\mathbb{R}^{n \times d}$ per obtenir com a resultat un valor pertanyent a l'espai $\mathbb{R}^{n \times d_v}$.

Una manera d'ampliar les capacitats del sistema és aplicar diverses transformacions en paral·lel del mateix tipus, però amb superconjunts ($\mathbf{Q}, \mathbf{K}, \mathbf{V}$) diferents. Cadascun d'ells reporta un valor diferent situat en l'espai $\mathbb{R}^{n \times d_v}$.

A l'equació 5 es mostra l'equació utilitzada per calcular l'atenció múltiple.

$$\mathbf{M} = \text{Concat}_{i=1}^H [\mathbf{D}_i(\mathbf{Q}_i, \mathbf{K}_i, \mathbf{V}_i)] \mathbf{W}_O \quad (5)$$

Si el nombre de blocs d'atenció paral·lels H es tria de manera tal que $H = \frac{d}{d_v}$ llavors les dimensions de l'espai de sortida seran iguals a les de l'espai d'entrada $\mathbb{R}^{n \times d}$. $\mathbf{W}_O$ és una matriu que realitza una transformació lineal posterior a la concatenació i permetria eventualment la modificació de les dimensions de l'espai de sortida, en cas que en el disseny pogués ser d'interès.

El mecanisme d'atenció múltiple permet aplicar en paral·lel transformacions diverses sobre els parells d'atributs d'entrada, augmentant la diversitat i complexitat de les comparacions, així com mantenir la dimensió de l'espai de treball, fet que pot ser d'interès en arquitectures que apilin blocs de la mateixa naturalesa.

L'algorisme 3 mostra la versió algorítmica del procés.

Algorisme 3 $\hat{\mathbf{V}} \leftarrow \text{atencio_multipçaal}(\mathbf{X}, \mathbf{Z} | \mathbf{W}_{qkv}, \text{Mask})$ (versió matricial)

/* Calcula l'atenció multipçaal (amb màscara) */

Entrada: $\mathbf{X} \in \mathbb{R}^{d_x \times \ell_x}$ i $\mathbf{Z} \in \mathbb{R}^{d_z \times \ell_z}$, representacions vectorials de les seqüències principal i de context

Sortida: $\hat{\mathbf{V}} \in \mathbb{R}^{d_{out} \times \ell_x}$, representació vectorial actualitzada de $\mathbf{X}$, recollint la informació de la seqüència $\mathbf{Z}$

Paràmetres: $\mathbf{W}_{qkv}$, consistent en: $\mathbf{W}_q \in \mathbb{R}^{d_{attn} \times d_x}$, $\mathbf{b}_q \in \mathbb{R}^{d_{attn}}$; $\mathbf{W}_k \in \mathbb{R}^{d_{attn} \times d_z}$, $\mathbf{b}_k \in \mathbb{R}^{d_{attn}}$; $\mathbf{W}_v \in \mathbb{R}^{d_{out} \times d_z}$, $\mathbf{b}_v \in \mathbb{R}^{d_{out}}$.

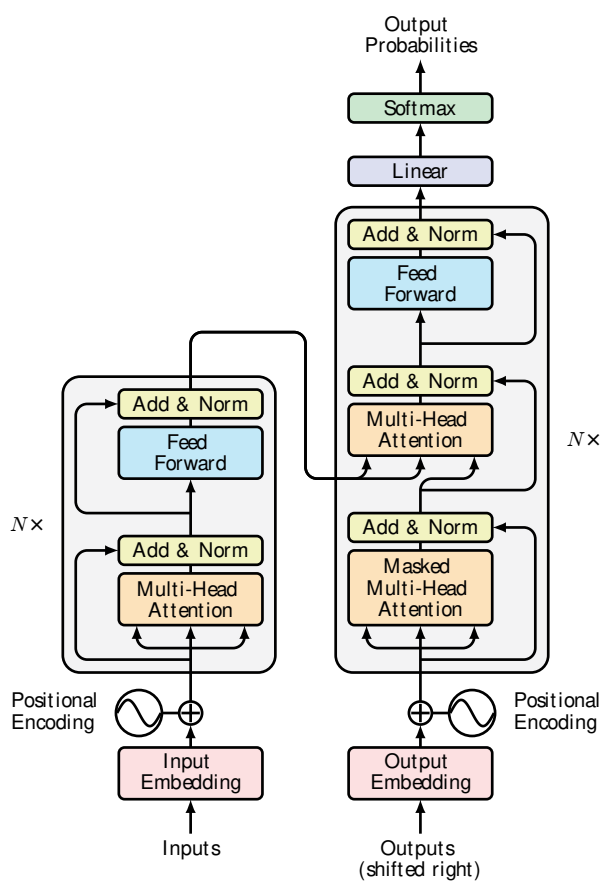
Hiperparàmetres: $\text{Mask} \in \{0, 1\}^{\ell_z \times \ell_x}$

- 1) Per a $h \in [H]$:
 - 2) $\mathbf{Y}^h \leftarrow \text{atencio}(\mathbf{X}, \mathbf{Z} | \mathbf{W}_{qkv}^h, \text{Mask})$
 - 3) $\mathbf{Y}^h \leftarrow [\mathbf{Y}^1; \mathbf{Y}^2; \dots; \mathbf{Y}^H]$
 - 4) return $\hat{\mathbf{V}} = \mathbf{W}_O \mathbf{Y} + \mathbf{b}_O \mathbf{1}^T$
-

2.2. Transformadors

Arribats a aquest punt, estem en condicions de presentar l'arquitectura del **transformador**. Com ja hem indicat amb anterioritat, aquest nou paradigma de disseny de xarxa neuronal es va introduir amb la publicació de Vaswani i altres (2017) i es va plantejar per a la seva aplicació en sistemes de traducció automàtica.

Figura 3. Arquitectura del transformador introduïda a «Attention is all you need» (Vaswani i altres, 2017)



A la figura 3 es mostra el diagrama dels blocs integrants de la xarxa de transformadors tal com va ser publicat per primera vegada a Vaswani i altres (2017). Cada element de la seqüència d'entrada es converteix en una representació numèrica (*input embedding*) per a posteriorment combinar-se (suma en aquest cas) amb una funció que codifica la posició de l'element en la seqüència (*positional encoding*). Aquest valor serveix d'entrada a una pila de N blocs denominats *codificadors*. Els atributs fruit de la codificació seran una de les entrades que alimentaran la xarxa de **descodificadors**. L'objectiu d'aquesta xarxa és predir la següent paraula i, per a això, utilitza no únicament la informació procedent de la cadena de l'entrada a través del codificador, sinó també tots els elements de la cadena de sortida produïts fins al moment.

Als subapartats següents descriurem amb detall cadascuna de les parts integrants de l'arquitectura, agrupant-les en els seus elements constitutius de major nivell.

2.2.1. Notació

Sigui V un conjunt finit denominat **vocabulari**, denotat com $[N_V] := \{1, \dots, N_V\}$. Pot estar format per lletres o paraules completes, encara que típicament està format per trossos de paraules denominats *tokens*.

Sigui $\mathbf{x} \equiv x[1 : \ell] \equiv x[1]x[2]...x[\ell] \in V^*$ una seqüència de *tokens*, per exemple, una frase, un paràgraf o un document.

Seguint la notació matemàtica i en contra del que s'estableix en alguns llenguatges de programació com ara C o Python, el primer índex de matrius i vectors utilitzats en aquest mòdul serà l'u. $X[1 : \ell]$, per exemple, refereix a la cadena de caràcters que va des del primer fins a l'element ℓ , tots dos inclosos.

Per a una matriu $M \in \mathbb{R}^{d \times d'}$, escrivim $M[i, :] \in \mathbb{R}^{d'}$ per a referir-nos a la fila i i $M[:, j] \in \mathbb{R}^d$ per a la columna j .

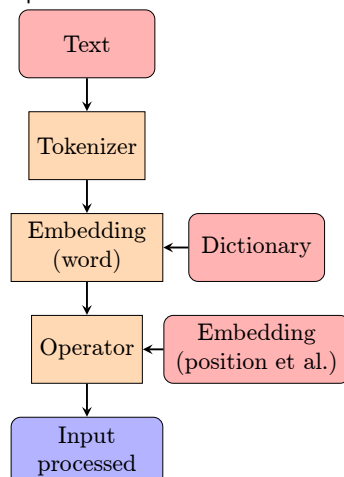
Fem servir la convenció matriu $\times$ columna més comuna en matemàtiques en comptes de fila $\times$ matriu més típica de la majoria de la literatura de transformadors, això és, les matrius estan transposades.

2.2.2. Elements constitutius del transformador

Tractament d'entrada

A la figura 4 es mostra el procés de tractament d'entrada propi del transformador. El text s'alimenta a un *tokenitzador*, la funció del qual és particionar la seqüència en els seus elements constituents. Cada element és representat per un vector ortogonal que té un 1 a la posició del diccionari identificativa de l'element i zeros a la resta. A continuació es fa una transformació lineal per comprimir la representació inicial en un vector dens de dimensió menor. Finalment, mitjançant un operador, que sol ser la suma (però res impedeix que pogués ser per exemple la concatenació o un altre de diferent), s'afegeix la informació referent a la posició que ocupa l'element dins de la seqüència. La representació numèrica de la seqüència ja està llesta per ser alimentada als blocs de processament següents.

Figura 4. Esquema típic de les operacions necessàries per convertir una entrada de text al format necessari per ser tractat pel transformador



La capa d'entrada s'encarrega del preprocessament de les dades per adequar-les a un format que sigui interpretable pel transformador. Els transformadors es van dissenyar inicialment com a forma alternativa de tractament de les dades seqüencials. El primer pas per al tractament d'aquesta mena de dades és la partició de la seqüència en els seus elements constitutius. Aquest procés és conegut habitualment com a *tokenització*. Aquesta partició es pot fer de moltes maneres: a nivell de paraula, parts de paraula o bé caràcters.

En els dissenys actuals de transformadors, el més habitual és utilitzar particions a nivell de parts de paraula. Moltes paraules estan constituïdes per diferents monemes (unitats mínimes de significat). Particionar a nivell de paraula té l'inconvenient que moltes de les unitats elementals de significat no se separen. Això sol ser un inconvenient que dificulta el funcionament de models interpretatius del llenguatge.

D'entrada es podria conjecturar que una separació a nivell de caràcter permetria al model reconstruir les unitats mínimes de significat. Els articles publicats indiquen explícitament que els resultats pràctics, almenys fins al moment, indiquen que la partició a nivell de caràcter és massa agressiva, amb resultats significativament inferiors als obtinguts utilitzant particions a nivell de parts de paraula.

Entre els particionadors més típics utilitzats amb transformadors tenim *BPE* (*Byte Pair Encoding*) (Gage, 1994), *Unigram* (Kudo, 2018) i *WordPiece* (Devlin i altres, 2018). La diferència existent entre aquests es basa en l'estratègia seguida per triar els parells de caràcters a concatenar per triar cadascuna de les subparaules per formar el diccionari.

Cada model preentrenat està dissenyat per ser utilitzat amb un diccionari específic derivat del particionador triat.

Per al cas dels transformadors per al tractament d'imatge, la proposta existent fins al moment consisteix a particionar la imatge en trossos no superposats de dimensions reduïdes i alimentar-los seqüencialment.

Si bé la preparació de les entrades descrites són les maneres actuals establertes de fer-ho, no hi ha una raó perquè això no pugui ser d'una altra manera. Per tant, és possible que en el futur puguin sorgir maneres alternatives de codificar les entrades.

Algorisme 4 *Embedding* d'un *token*

Entrada: $v \in V \cong [N_V]$, identificador del *token* (p.e. one-hot)

Sortida: $e \in \mathbb{R}^{d_e}$, la representació vectorial d'un *token*

Paràmetres: $W_e \in \mathbb{R}^{d_e \times N_V}$, la matriu d'*embedding*

retorna: $e = W_e[:, v]$

Algorisme 5 *Embedding* de la posició

Entrada: $\ell \in \ell_{max}$, posició del *token* dins de la seqüència

Sortida: $\mathbf{e}_p \in \mathbb{R}^{d_e}$, la representació vectorial de la posició

Paràmetres: $\mathbf{W}_p \in \mathbb{R}^{d_e \times \ell_{max}}$, la matriu d'*embeddings* de posició

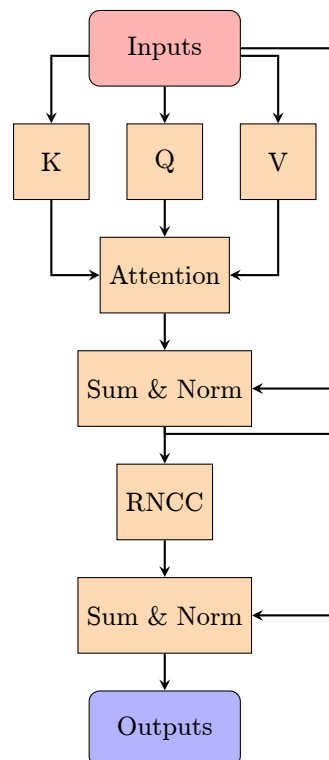
retorna: $\mathbf{e}_p = \mathbf{W}_p[:, \ell]$

Codificadors

El codificador està integrat per un capçal d'autoatenció múltiple seguit d'una xarxa completament connectada. Totes dues capes formen part d'una xarxa residual i, per tant, integren a continuació de cadascuna d'aquestes dues capes la suma de l'entrada i la seva normalització.

A la figura 5 es mostra el diagrama de blocs dels elements integrants d'un codificador.

Figura 5. Esquema típic dels elements integrants d'un codificador. La xarxa de codificació integra diversos d'aquests elements en sèrie. La sortida d'un serveix d'entrada per al següent.



Descodificadors

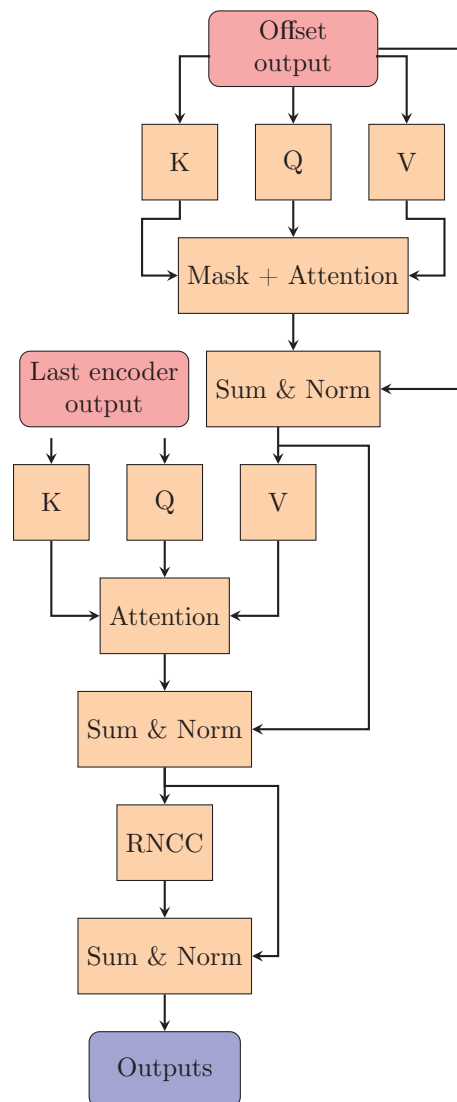
El descodificador és un bloc integrat per un capçal d'autoatenció múltiple que rep com a entrades els elements de la cadena de sortida de la xarxa del transformador desplaçats una posició. Per assegurar que la predicció es realitza

usant únicament la informació precedent, però no la posterior, integra una matriu d'emmaskat que permet anul·lar l'efecte dels elements posteriors de la cadena de sortida.

A continuació d'aquest primer bloc, n'hi ha un segon que rep com a entrada la sortida del primer, així com la sortida procedent de la codificació de la cadena d'entrada. La sortida d'aquest bloc s'alimenta a una capa completament connectada. Totes aquestes etapes són també capes residuals i estan seguides de la seva corresponent suma amb l'entrada i normalització. La xarxa de descodificació apila en sèrie N elements d'aquest tipus.

A la figura 6 es mostra el diagrama de blocs dels elements integrants d'un descodificador.

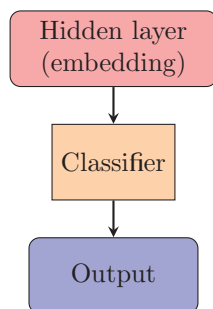
Figura 6. Esquema típic dels elements integrants d'un descodificador. La xarxa de descodificació integra diversos d'aquests elements en sèrie. La sortida d'un serveix d'entrada per al següent. A tots ells s'alimenta la sortida de la xarxa de codificadors.



Tractament de sortida

A la capa de sortida cal realitzar una conversió des de l'espai de representació interna fins al de representació externa (vegeu la figura 7). És la transformació inversa a l'operació d'*embedding*, on passem del vector representatiu del diccionari a una representació interna.

Figura 7. Esquema típic de les operacions necessàries per a convertir l'última capa oculta del transformador en una sortida



Hi ha dues maneres de dur a terme la conversió: la primera és tractar la capa com a independent, realitzant l'aprenentatge dels pesos de conversió, mentre que la segona és utilitzar la matriu inversa de l'*embedding*, sense realitzar un aprenentatge independent.

A Vaswani i altres (2017) s'utilitza l'estratègia definida a Press y Wolf (2016) de lligar la transformació de la capa d'entrada a la capa de sortida. Un dels passos inicials per preparar les entrades implica convertir la representació *one-hot* de la seva posició al diccionari per una representació «densa» de les dimensions del model. Aquest pas es fa mitjançant una transformació lineal que redueix les dimensions des de les del diccionari a la interna de treball del model. La idea presentada a Press y Wolf (2016) i utilitzada posteriorment a Vaswani i altres (2017) implica utilitzar la mateixa matriu de transformació tant per a les entrades com per a les sortides. Sent això així, el classificador de sortida passa a ser una transformació lineal amb els mateixos pesos que la de l'entrada.

L'algorisme 6 mostra la versió algorítmica del procés.

Algorisme 6 *Embedding* invers

/* Converteix la representació interna de la xarxa en un element */

Entrada: $\mathbf{e} \in \mathbb{R}^{d_e}$, representació interna d'un element

Sortida: $\mathbf{p} \in \Delta(V)$, vector que conté la representació distribució de probabilitat sobre el conjunt d'elements integrants del vocabulari

Paràmetres: $\mathbf{W}_u \in \mathbb{R}^{N_V \times d_e}$, matriu d'*embedding* inversa

return $\mathbf{p} = \text{softmax}(\mathbf{W}_u \mathbf{e})$

Normalització de capes

Les capes de normalització estan presents com a bloc constituent tant en els codificadors com en els descodificadors. La seva finalitat és el control explícit de la mitjana i la variància de l'activació de cada neurona.

L'algorisme 7 mostra de forma detallada el procés que té lloc en aquestes capes.

Algorisme 7 $\hat{\mathbf{e}} \leftarrow \text{normalitzacio_de_capa}(\mathbf{e}|\boldsymbol{\gamma},\boldsymbol{\beta})$

/ Normalitza les activacions $\mathbf{e}$ de la capa */*

Entrada: $\mathbf{e} \in \mathbb{R}^{d_e}$, activacions d'entrada sense normalitzar

Sortida: $\hat{\mathbf{e}} \in \mathbb{R}^{d_e}$, activacions normalitzades

Paràmetres: $\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^{d_e}$, vector d'escalat i desplaçament

- 1) $\mathbf{m} \leftarrow \sum_{i=1}^{d_e} \mathbf{e}[i] / d_e$
 - 2) $v \leftarrow \sum_{i=1}^{d_e} (\mathbf{e}[i] - \mathbf{m})^2 / d_e$
 - 3) **return** $\hat{\mathbf{e}} = \frac{\mathbf{e} - \mathbf{m}}{\sqrt{v}} \odot \boldsymbol{\gamma} + \boldsymbol{\beta}$, on $\odot$ denota multiplicació component-component
-

2.3. Tipus de transformadors

Al subapartat anterior s'ha introduït l'arquitectura del transformador mostrant el diagrama original introduït a Vaswani i altres (2017). Com ja hem indicat amb anterioritat, la proposta inicial es va fer com a mitjà alternatiu de resolució de problemes de traducció de seqüència a seqüència i representa el primer dels tipus de models possibles de transformadors. En aquest subapartat presentarem modificacions de l'arquitectura original que van sorgir posteriorment i que han donat lloc a subfamílies de transformadors especialitzades en la resolució de problemes diferents. Actualment podem distingir entre tres tipus de models o arquitectures: els autoregressius, els d'autocodificació i els de conversió de seqüència a seqüència.

2.3.1. Transformadors seqüència a seqüència

Els models de seqüència a seqüència utilitzen dues unitats bàsiques de disseny denominades codificadors i descodificadors. La seva finalitat és la conversió d'una seqüència d'entrada en una altra de diferent. Les seves aplicacions principals estan relacionades amb la traducció, el resum i la resposta a preguntes. El model original és un exemple d'aquesta mena d'aplicacions. T5 (Raffel i altres, 2020) seria un exemple presentat després de l'article general de la mateixa arquitectura utilitzada per resoldre un altre tipus de tasques més específiques.

L'algorisme 8 recull els detalls d'implementació del que seria un transformador de seqüència a seqüència.

Algorisme 8 P $\leftarrow \text{transformadorCD}(\mathbf{x} \mid \boldsymbol{\theta})$

/* Transformador codificador/descodificador o de seqüència a seqüència */

Entrada: $\mathbf{z}, \mathbf{x} \in V^*$, dues seqüències d'identificadors de *tokens*Sortida: $\mathbf{P} \in (0,1)^{N_V \times \text{longitud}(\mathbf{x})}$, on la columna t de $\mathbf{P}$ representa $\hat{P}_{\boldsymbol{\theta}}(\mathbf{x}[t+1] \mid \mathbf{x}[1:t], \mathbf{z})$ Hiperparàmetres: $\ell_{\max}, L_{\text{enc}}, L_{\text{dec}}, H, d_e, d_{\text{mlp}}, d_f \in \mathbb{N}$ Paràmetres: $\boldsymbol{\theta}$ que inclou: $\mathbf{W}_e \in \mathbb{R}^{d_e \times N_V}, \mathbf{W}_p \in \mathbb{R}^{d_e \times \ell_{\max}}$, matrius d'*embedding* de *token* i posicióPer a $l \in [L_{\text{enc}}]$: $\mathbf{W}_{\text{qkv},l}^{\text{enc}}$, autoatenció de la capa l $\gamma_l^1, \beta_l^1, \gamma_l^2, \beta_l^2 \in \mathbb{R}^{d_e}$, dos conjunts de normalització de capes $\mathbf{W}_{\text{mlp}1}^l \in \mathbb{R}^{d_{\text{mlp}} \times d_e}, \mathbf{b}_{\text{mlp}1}^l \in \mathbb{R}^{d_{\text{mlp}}}, \mathbf{W}_{\text{mlp}2}^l \in \mathbb{R}^{d_{\text{mlp}} \times d_e}, \mathbf{b}_{\text{mlp}2}^l \in \mathbb{R}^{d_{\text{mlp}}}$ Per a $l \in [L_{\text{dec}}]$: $\mathbf{W}_{\text{qkv},l}^{\text{dec}}, \mathbf{W}_{\text{qkv},l}^{\text{ed}}$, autoatenció i autoatenció creuada de la capa l $\gamma_l^3, \beta_l^3, \gamma_l^4, \beta_l^4, \gamma_l^5, \beta_l^5 \in \mathbb{R}^{d_e}$, tres conjunts de normalització de capes $\mathbf{W}_{\text{mlp}3}^l \in \mathbb{R}^{d_{\text{mlp}} \times d_e}, \mathbf{b}_{\text{mlp}3}^l \in \mathbb{R}^{d_{\text{mlp}}}, \mathbf{W}_{\text{mlp}4}^l \in \mathbb{R}^{d_{\text{mlp}} \times d_e}, \mathbf{b}_{\text{mlp}4}^l \in \mathbb{R}^{d_{\text{mlp}}}$ $\mathbf{W}_u \in \mathbb{R}^{N_V \times d_e}$, matriu inversa d'*embedding*

/* Codificació de la seqüència de context */

1 $\ell_z \leftarrow \text{longitud}(\mathbf{z})$ 2 per a $t \in [\ell_z] : \mathbf{e}_t \leftarrow \mathbf{W}_e[:, \mathbf{z}[t]] + \mathbf{W}_p[:, t]$ 3 $\mathbf{Z} \leftarrow [\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_{\ell_z}]$ 4 per a $\ell = 1, 2, \dots, L_{\text{enc}}$ fer5 $\mathbf{Z} \leftarrow \mathbf{Z} + \text{atencio_multicapçal}(\mathbf{Z} \mid \mathbf{W}_{\text{qkv},\ell}^{\text{enc}}, \text{Mask} \equiv 1)$ 6 per a $t \in [\ell_z] : \mathbf{Z}[:, t] \leftarrow \text{normalitzacio}(\mathbf{Z}[:, t] \mid \gamma_t^1, \beta_t^1)$ 7 $\mathbf{Z} \leftarrow \mathbf{Z} + \mathbf{W}_{\text{mlp}2}^l \text{ReLU}(\mathbf{W}_{\text{mlp}1}^l \mathbf{Z} + \mathbf{b}_{\text{mlp}1}^l \mathbf{1}^T) + \mathbf{b}_{\text{mlp}2}^l \mathbf{1}^T$ 8 per a $t \in [\ell_z] : \mathbf{Z}[:, t] \leftarrow \text{normalitzacio}(\mathbf{Z}[:, t] \mid \gamma_t^2, \beta_t^2)$

9 fper

/* Descodificació de la primera seqüència condicionada pel context */

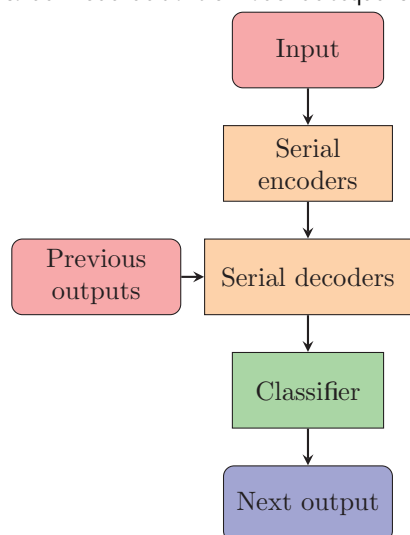
10 $\ell_x \leftarrow \text{longitud}(\mathbf{x})$ 11 per a $t \in [\ell_x] : \mathbf{e}_t \leftarrow \mathbf{W}_e[:, \mathbf{x}[t]] + \mathbf{W}_p[:, t]$ 12 $\mathbf{X} \leftarrow [\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_{\ell_x}]$ 13 per a $i = 1, 2, \dots, L_{\text{dec}}$ fer14 $\mathbf{X} \leftarrow \mathbf{X} + \text{atencio_multicapçal}(\mathbf{X} \mid \mathbf{W}_{\text{qkv},i}^{\text{dec}}, \text{Mask}[t, t'] = [[t \leq t']])$ 15 per a $t \in [\ell_x] : \mathbf{X}[:, t] \leftarrow \text{normalitzacio}(\mathbf{X}[:, t] \mid \gamma_t^3, \beta_t^3)$ 16 $\mathbf{X} \leftarrow \mathbf{X} + \text{atencio_multicapçal}(\mathbf{X}, \mathbf{Z} \mid \mathbf{W}_{\text{qkv},i}^{\text{ed}}, \text{Mask} \equiv 1)$ 17 per a $t \in [\ell_x] : \mathbf{X}[:, t] \leftarrow \text{normalitzacio}(\mathbf{X}[:, t] \mid \gamma_t^4, \beta_t^4)$ 18 $\mathbf{X} \leftarrow \mathbf{X} + \mathbf{W}_{\text{mlp}4}^l \text{ReLU}(\mathbf{W}_{\text{mlp}3}^l \mathbf{X} + \mathbf{b}_{\text{mlp}3}^l \mathbf{1}^T) + \mathbf{b}_{\text{mlp}4}^l \mathbf{1}^T$

19 fper

20 return $P = \text{softmax}(\mathbf{W}_u \mathbf{X})$

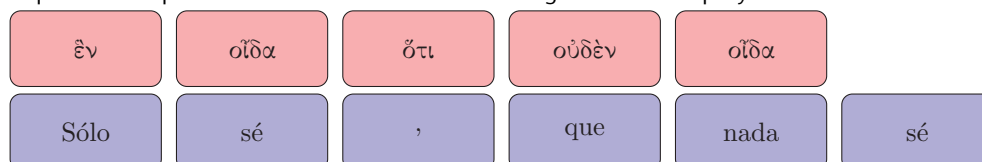
A la figura 8 es representa el diagrama funcional típic representatiu d'aquesta arquitectura. Aquest diagrama és equivalent al de la figura 3, mostrant els elements constitutius al més alt nivell, és a dir: els blocs de tractament de la seqüència d'entrada, els codificadors, decodificadors i finalment la capa de classificació i sortida.

Figura 8. Arquitectura típica del model de transformador de seqüència a seqüència



A la figura 9 mostrem un exemple d'una seqüència típica d'entrada i una altra de sortida. Les seqüències mostrades serien un exemple d'una traducció d'una seqüència inicial en grec clàssic a una altra seqüència en espanyol. Al model s'entraria la primera i s'obtindria com a sortida la segona.

Figura 9. Exemple d'un conjunt d'entrada i sortida comuna d'un model seqüència a seqüència. En aquest cas seria un model de traducció de grec clàssic a l'espanyol.



A l'algorisme 9 es mostra el que seria el codi necessari per realitzar l'optimització del model utilitzant un conjunt de dades d'entrada.

Algorisme 9 $\hat{\theta} \leftarrow \text{entrenamentCD}(\mathbf{z}_{1:N_{data}}, \mathbf{x}_{1:N_{data}}, \theta)$

/* Entrenament d'un transformador de seqüència a seqüència */

Entrada: $(\mathbf{z}_n, \mathbf{x}_n)_{n=1}^{N_{data}}$, un conjunt de dades de parelles de seqüències**Entrada:** θ , valors de partida dels paràmetres del transformador**Sortida:** $\hat{\theta}$, valor dels paràmetres després de l'entrenament**Hiperparàmetres:** $N_{epochs} \in \mathbb{N}, \eta \in (0, \infty)$

```

1  per a  $i = 1, 2, \dots, N_{epochs}$  fer
2      per a  $n = 1, 2, \dots, N_{data}$  fer
3           $\ell \leftarrow \text{longitud}(\mathbf{x}_n)$ 
4           $\mathbf{P}_\theta \leftarrow \text{transformador\_de\_codificadors\_descodificadors}(\mathbf{z}_n, \mathbf{x}_n \mid \theta)$ 
5           $\text{loss}(\theta) = -\sum_{t=1}^{\ell-1} \log P(\theta)[x_n[t+1], t]$ 
6           $\theta \leftarrow \theta - \eta \nabla \text{loss}(\theta)$ 
7      fper
8  fper
9  retorna:  $\hat{\theta} = \theta$ 

```

2.3.2. Transformadors bidireccionals o d'autocodificació

Els models d'autocodificació són aquells que estan preentrenats per a la correcció d'entrades corrompudes artificialment. Això es fa per entrenar el model en la reconstrucció del text original. Aquest tipus de models es correspon amb la part del codificador de l'arquitectura original en el sentit que no utilitzen màscares per anul·lar l'efecte del text posterior. Aquests models construeixen una representació bidireccional de la frase íntegrament i poden ser posteriorment entrenats per a la resolució de tasques específiques com la generació de text, encara que la seva aplicació principal és la classificació de frases. Un exemple típic d'aquests models seria BERT (Devlin i altres, 2018).

L'algorisme 10 recull els detalls d'implementació del que seria un transformador bidireccional, això és, un transformador format únicament per codificadors.

A l'algorisme 11 es mostra el que seria el codi necessari per realitzar l'optimització del model utilitzant un conjunt de dades d'entrada.

Algorisme 10 $\mathbf{P} \leftarrow \text{codificador}(\mathbf{x} \mid \boldsymbol{\theta})$

/* Transformador format per codificadors, bidireccional o autocodificador */

Entrada: $\mathbf{x} \in V^*$, una seqüència d'identificadors de *tokens***Sortida:** $\mathbf{P} \in (0,1)^{N_V \times \text{longitud}(\mathbf{x})}$, on cada columna de $\mathbf{P}$ és una distribució sobre tot el vocabulari**Hiperparàmetres:** $\ell_{\max}, L, H, d_e, d_{\text{mlp}}, d_f \in \mathbb{N}$ **Paràmetres:** $\boldsymbol{\theta}$ que inclou a: $\mathbf{W}_e \in \mathbb{R}^{d_e \times N_V}, \mathbf{W}_p \in \mathbb{R}^{d_e \times \ell_{\max}}$, matrius d'*embedding* de *token* i posicióPer a $l \in [L]$: $\mathbf{W}_{qkv,l}$, paràmetres d'autoatenció de la capa l $\gamma_l^1, \beta_l^1, \gamma_l^2, \beta_l^2 \in \mathbb{R}^{d_e}$, dos conjunts de normalització de capes $\mathbf{W}_{\text{mlp}1}^l \in \mathbb{R}^{d_{\text{mlp}} \times d_e}, \mathbf{b}_{\text{mlp}1}^l \in \mathbb{R}^{d_{\text{mlp}}}, \mathbf{W}_{\text{mlp}2}^l \in \mathbb{R}^{d_{\text{mlp}} \times d_e}, \mathbf{b}_{\text{mlp}2}^l \in \mathbb{R}^{d_{\text{mlp}}}$ $\mathbf{W}_f \in \mathbb{R}^{d_f \times d_e}, \mathbf{b}_f \in \mathbb{R}^{d_f}, \gamma, \beta \in \mathbb{R}^{d_f}$, projecció lineal i normalització final $\mathbf{W}_u \in \mathbb{R}^{N_V \times d_e}$, matriu inversa d'*embedding*

- 1 $\ell \leftarrow \text{longitud}(\mathbf{x})$
- 2 per a $t \in [\ell] : \mathbf{e}_t \leftarrow \mathbf{W}_e[:, x[t]] + \mathbf{W}_p[:, t]$
- 3 $\mathbf{X} \leftarrow [\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_\ell]$
- 4 per a $\ell = 1, 2, \dots, L$ fer
 - 5 $\mathbf{X} \leftarrow \mathbf{X} + \text{atencio_multicapçal}(\mathbf{X} \mid \mathbf{W}_{qkv,l}, \text{Mask} \equiv 1)$
 - 6 per a $t \in [\ell] : \mathbf{X}[:, t] \leftarrow \text{normalitzacio}(\mathbf{X}[:, t] \mid \gamma_l^1, \beta_l^1)$
 - 7 $\mathbf{X} \leftarrow \mathbf{X} + \mathbf{W}_{\text{mlp}1}^l \text{GELU}(\mathbf{W}_{\text{mlp}1}^l \mathbf{X} + \mathbf{b}_{\text{mlp}1}^l \mathbf{1}^T) + \mathbf{W}_{\text{mlp}2}^l \mathbf{1}^T$
 - 8 per a $t \in [\ell] : \mathbf{X}[:, t] \leftarrow \text{normalitzacio}(\mathbf{X}[:, t] \mid \gamma_l^2, \beta_l^2)$
 - 9 fper
 - 10 $\mathbf{X} \leftarrow \text{GELU}(\mathbf{W}_f \mathbf{X} + \mathbf{b}_f \mathbf{1}^T)$
 - 11 per a $t \in [\ell] : \mathbf{X}[:, t] \leftarrow \text{normalitzacio}(\mathbf{X}[:, t] \mid \gamma, \beta)$
 - 12 return $\mathbf{P} = \text{softmax}(\mathbf{W}_u \mathbf{X})$

Algorisme 11 $\hat{\theta} \leftarrow \text{entrenamentC}(\mathbf{x}_{1:N_{data}}, \theta)$

/* Entrenament mitjançant modelització mitjançant màscara */

Entrada: $\{\mathbf{x}_n\}_{n=1}^{N_{data}}$, un conjunt de dades de seqüències**Entrada:** θ , valors de partida dels paràmetres del transformador**Sortida:** $\hat{\theta}$, valor dels paràmetres després de l'entrenament**Hiperparàmetres:** $N_{epochs} \in \mathbb{N}, \eta \in (0, \infty), p_{mask} \in (0, 1)$

```

1  per a  $i = 1, 2, \dots, N_{epochs}$  fer
2      per a  $n = 1, 2, \dots, N_{data}$  fer
3           $\ell \leftarrow \text{longitud}(\mathbf{x}_n)$ 
4          per a  $t = 1, 2, \dots, \ell$  fer
5               $\tilde{x}_n[t] \leftarrow \text{emascara\_token o } x_n[t] \text{ aleatòriament amb}$ 
                  probabilitat  $p_{mask}$  o  $1 - p_{mask}$ 
6          fper
7           $\tilde{T} \leftarrow \{t \in [\ell] : \tilde{x}_n[t] = \text{emascara\_token}\}$ 
8           $\mathbf{P}(\theta) \leftarrow \text{transformador\_de\_codificadors}(\tilde{\mathbf{x}}_n \mid \theta)$ 
10          $\text{loss}(\theta) = -\sum_{t \in \tilde{T}} \log P(\theta)[x_n[t], t]$ 
11          $\theta \leftarrow \theta - \eta \nabla \text{loss}(\theta)$ 
12     fper
13 fper
14 retorna:  $\hat{\theta} = \theta$ 

```

La xarxa de codificació apila en sèrie L elements del mateix tipus. La sortida de cada bloc serveix d'entrada per al següent. Les dimensions de la transformació es mantenen, per la qual cosa entrada i sortida tenen la mateixa mida.

Figura 10. Arquitectura típica del model de transformador autocodificador

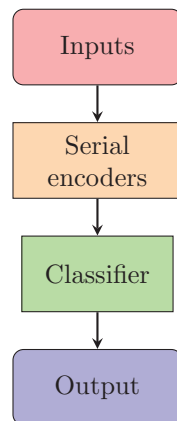
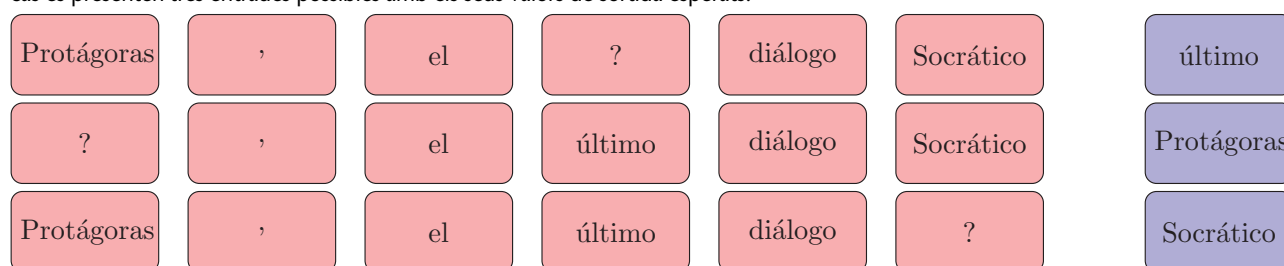


Figura 11. Exemple d'entrades i sortides típiques dels models d'autocodificació. En aquest cas es presenten tres entrades possibles amb els seus valors de sortida esperats.



Taula 1. Característiques d'alguns dels primers models bidireccionals més coneguts

	BERT-b	BERT-l	XLNet-l	RoBERTa-l	DistillBERT-b	ALBERT-l
Arquitectura						
n_{param}	108×10^6	340×10^6	340×10^6	355×10^6	66×10^6	18×10^6
T_{param}	432 MB	1360 MB	1360 MB	1420MB	264MB	72 MB
ℓ_{max}	512	512				
L_{enc}	12	24	24	24	6	24
H	12	16	16	16	12	16
d_{attn}	64	64	64	64		8
d_e	768	1024	1024	1024		128
d_{mlp}	3072	4096		4096		512
N_V	30522					
$C_{entrada}$	WordPiece					
Entrenament						
T_{dades}	16 GB	16 GB	113 GB	160 GB	16 GB	16 GB
n_{tokens}	3.3×10^9	3.3×10^9	33×10^9		3.3×10^9	
$ratio_{compr}$	30.5	9.7	97.0		50.0	
Optimitzador				Adam		
BS				8000		
Rendiment						
GLUE		base	+15%	+20%	-3%	+25%

2.3.3. Transformadors autoregressius

Els models autoregressius són aquells que a partir d'una seqüència d'entrada inicial generen noves sortides que al seu torn s'incorporen a la seqüència inicial com a entrada per generar noves sortides. La seva principal aplicació és la generació de text, encara que és possible ajustar-los amb un entrenament posterior per adaptar-los a la resolució de problemes específics. Un exemple d'aquesta mena de models seria la família de GPT.

L'algorisme 12 recull els detalls d'implementació del que seria un transformador autoregressiu, és a dir, un transformador format únicament per descodificadors.

N_V

La mida del vocabulari depèn de les característiques del model. Aquest valor variarà depenent de si s'ha entrenat per ser sensible majúscules, per a un únic idioma o bé per a diferents llenguatges. És un valor orientatiu corresponent a un model preentrenat sense especificar, amb objectiu únicament orientatiu.

Algorisme 12 $P \leftarrow \text{transformadorD}(\mathbf{x} \mid \boldsymbol{\theta})$

/* Transformador format per descodificadors o autoregressiu */

Entrada: $\mathbf{x} \in V^*$, una seqüència d'identificadors de *tokens***Sortida:** $\mathbf{P} \in (0,1)^{N_V \times \text{longitud}(\mathbf{x})}$, on la columna t de $\mathbf{P}$ representa $\hat{P}_{\boldsymbol{\theta}}(\mathbf{x}[t+1] \mid \mathbf{x}[1:t])$ **Hiperparàmetres:** $\ell_{\max}, L, H, d_e, d_{mlp}, d_f \in \mathbb{N}$ **Paràmetres:** $\boldsymbol{\theta}$ que inclou a: $\mathbf{W}_e \in \mathbb{R}^{d_e \times N_V}, \mathbf{W}_p \in \mathbb{R}^{d_e \times \ell_{\max}}$, matrius d'*embedding* de *token* i posicióPer a $l \in [L]$: $\mathbf{W}_{qkv,l}$, paràmetres d'autoatenció de la capa l $\gamma_l^1, \beta_l^1, \gamma_l^2, \beta_l^2 \in \mathbb{R}^{d_e}$, dos conjunts de normalització de capes $\mathbf{W}_{mlp1}^l \in \mathbb{R}^{d_{mlp} \times d_e}, \mathbf{b}_{mlp1}^l \in \mathbb{R}^{d_{mlp}}, \mathbf{W}_{mlp2}^l \in \mathbb{R}^{d_{mlp} \times d_e}, \mathbf{b}_{mlp2}^l \in \mathbb{R}^{d_{mlp}}$ $\mathbf{W}_f \in \mathbb{R}^{d_f \times d_e}, \mathbf{b}_f \in \mathbb{R}^{d_f}, \gamma, \beta \in \mathbb{R}^{d_f}$, projecció lineal i normalització final $\mathbf{W}_u \in \mathbb{R}^{N_V \times d_e}$, matriu inversa d'*embedding*

- 1 $\ell \leftarrow \text{longitud}(\mathbf{x})$
- 2 per a $t \in [\ell] : \mathbf{e}_t \leftarrow \mathbf{W}_e[:, \mathbf{x}[t]] + \mathbf{W}_p[:, t]$
- 3 $\mathbf{X} \leftarrow [\mathbf{e}_1, \mathbf{e}_2, \dots, \mathbf{e}_\ell]$
- 4 per a $\ell = 1, 2, \dots, L$ fer
 - 5 per a $t \in [\ell] : \mathbf{X}[:, t] \leftarrow \text{normalitzacio}(\mathbf{X}[:, t] \mid \gamma_l^1, \beta_l^1)$
 - 6 $\mathbf{X} \leftarrow \mathbf{X} + \text{atencio_multicapçal}(\mathbf{X} \mid \mathbf{W}_{qkv,l}, \text{Mask}[t, t'] = [[t \leq t']])$
 - 7 per a $t \in [\ell] : \mathbf{X}[:, t] \leftarrow \text{normalitzacio}(\mathbf{X}[:, t] \mid \gamma_l^2, \beta_l^2)$
 - 8 $\mathbf{X} \leftarrow \mathbf{X} + \mathbf{W}_{mlp2}^l \text{GELU}(\mathbf{W}_{mlp1}^l \mathbf{X} + \mathbf{b}_{mlp1}^l \mathbf{1}^T) + \mathbf{b}_{mlp2}^l \mathbf{1}^T$
- 9 fper
- 10 per a $t \in [\ell] : \mathbf{X}[:, t] \leftarrow \text{normalitzacio}(\mathbf{X}[:, t] \mid \gamma, \beta)$
- 11 return $P = \text{softmax}(\mathbf{W}_u \mathbf{X})$

Igual que en els codificadors, la xarxa de descodificació apila en sèrie N elements del mateix tipus. La sortida de cada bloc serveix d'entrada per al següent. Les dimensions de la transformació es mantenen, per la qual cosa entrada i sortida tenen la mateixa mida.

Figura 12. Arquitectura de transformador autoregressiu

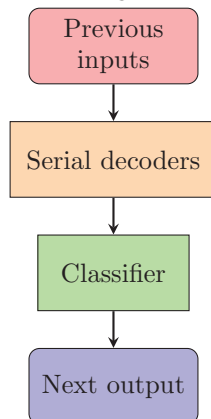
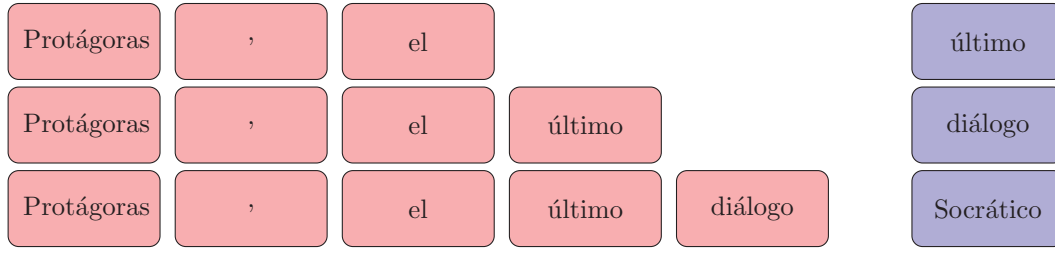


Figura 13. Exemple d'entrades i sortides típiques dels models autoregressius. En aquest cas es presenten tres entrades possibles amb les seves respectives sortides esperades.



Taula 2. Característiques dels models autoregressius de la família GPT.

	GPT-1	GPT-2	GPT-3
Arquitectura			
n_{param}	117×10^6	1.5×10^9	175×10^9
T_{param}	468 MB	6 GB	700GB
ℓ_{max}	512	1024	2048
L_{dec}	12	48	96
H	12	12	96
d_{attn}	64		128
d_{mlp}	3072		
N_V	40478	50257	50257
$c_{entrada}$	BPE	BPE	BPE
d_e	768	1600	12288
Entrenament			
T_{dades}	1 GB	40 GB	2 TB
n_{tokens}	25×10^7	10×10^9	499×10^9
$ratio_{compr}$	2.13	6.66	2.85
Optimitzador	Adam	Adam	Adam
BS	64	512	3.2×10^6

A l'algorisme 13 es mostra el conjunt d'operacions que es duen a terme en un bucle d'entrenament.

Algorisme 13 $\hat{\theta} \leftarrow \text{entrenamentD}(\mathbf{x}_{1:N_{data}}, \theta)$

/* Entrenament per a predicció de següent *token* */

Entrada: $\{\mathbf{x}_n\}_{n=1}^{N_{data}}$, un conjunt de dades de seqüències

Entrada: θ , valors de partida dels paràmetres del transformador

Sortida: $\hat{\theta}$, valor dels paràmetres després de l'entrenament

- 1 per a $i = 1, 2, \dots, N_{epochs}$ fer
 - 2 per a $n = 1, 2, \dots, N_{data}$ fer
 - 3 $\ell \leftarrow \text{longitud}(\mathbf{x}_n)$
 - 4 $\mathbf{P}_\theta \leftarrow \text{transformador_de_descodificadors}(\mathbf{x}_n \mid \theta)$
 - 5 $\text{loss}(\theta) = -\sum_{t=1}^{\ell-1} \log P(\theta)[x_n[t+1], t]$
 - 6 $\theta \leftarrow \theta - \eta \nabla \text{loss}(\theta)$
 - 7 fper
 - 8 fper
 - 9 retorna: $\hat{\theta} = \theta$
-

Algorisme 14 $\hat{\theta} \leftarrow \text{inferencia_transformadorD}(\mathbf{x}, \hat{\theta})$

/* Predicció utilitzant un model entrenat */

Entrada: $\hat{\theta}$, paràmetres entrenats**Entrada:** $\mathbf{x} \in V^*$, una seqüència d'entrada**Sortida:** $\gamma \in V^*$, predicció de la continuació de la seqüència d'entrada**Hiperparàmetres:** $\ell_{gen} \in \mathbb{N}, \tau \in (0, \infty)$

```

1   $\ell \leftarrow \text{longitud}(\mathbf{x})$ 
2  per a  $n = 1, 2, \dots, \ell_{gen}$  fer
3       $\mathbf{P} \leftarrow \text{transformador\_de\_descodificadors}(\mathbf{x} \mid \hat{\theta})$ 
4       $\mathbf{p} \leftarrow \mathbf{P}[:, \ell + i - 1]$ 
5      mostreja un token  $\gamma$  de  $\mathbf{q} \sim \mathbf{p}^{1/\tau}$ 
6       $\mathbf{x} \leftarrow [\mathbf{x}, \gamma]$ 
7  fper
8  retorna  $\gamma = \mathbf{x}[\ell + 1 : \ell + \ell_{gen}]$ 
```

Algorisme 15 $\hat{\theta} \leftarrow \text{inferencia_transformadorCD}(\mathbf{z}, \hat{\theta})$

/* Predicció usant un transformador seqüència a seqüència */

Entrada: $\hat{\theta}$, paràmetres entrenats**Entrada:** $\mathbf{z} \in V^*$, una seqüència d'entrada**Sortida:** $\hat{\mathbf{x}} \in V^*$, predicció de la seqüència de sortida**Hiperparàmetres:** $\tau \in (0, \infty)$

```

1   $\hat{\mathbf{x}} \leftarrow [\text{bos\_token}]$ 
2   $\gamma \leftarrow 0$ 
3  mentrestant  $\gamma \neq \text{eos\_token}$  fer
4       $\mathbf{P} \leftarrow \text{transformador\_CD}(\mathbf{z}, \hat{\mathbf{x}} \mid \hat{\theta})$ 
5       $\mathbf{p} \leftarrow \mathbf{P}[:, \text{longitud}(\hat{\mathbf{x}})]$ 
5      mostreja un token  $\gamma$  de  $\mathbf{q} \sim \mathbf{p}^{1/\tau}$ 
6       $\hat{\mathbf{x}} \leftarrow [\hat{\mathbf{x}}, \gamma]$ 
7  fper
8  retorna  $\hat{\mathbf{x}}$ 
```


2.4. Transformadors per a processament del llenguatge natural (NLP)

2.4.1. Desenvolupaments previs: *embeddings*

Previ a la introducció dels algorismes basats en transformadors és important fer esment a un conjunt d'algorismes que es van crear en els anys anteriors, concretament als algorismes de creació d'*embeddings*.

En el context de NLP, un *embedding* és una representació matemàtica del significat de les paraules. Aquesta transformació parteix del supòsit que és possible representar el coneixement en un espai vectorial multidimensional. D'aquesta manera, qualsevol paraula del llenguatge tindrà una transformació en aquest espai. Als vectors representatius de les paraules se'ls denomina *embeddings*.

En els anys anteriors a la introducció dels transformadors, es van realitzar multitud de publicacions que definien diferents mètodes per realitzar aquesta transformació. Entre els més coneguts tenim **Word2Vec** (Mikolov i altres, 2013), **GloVe** (Pennington i altres, 2014), **FastText** (Joulin i altres, 2016), entre altres.

Aquests algorismes fan la transformació utilitzant una xarxa neuronal poc profunda. Entre les propietats més destacades d'aquests algorismes estan les següents:

- **Similitud entre paraules sinònimes o semblants:** en l'espai transformat, paraules de significat similar convergeixen en ubicacions similars. Així, per exemple, paraules com 'cotxe', 'vehicle' o 'furgoneta' estan relativament més pròximes entre elles que d'altres com 'lluna', 'espai' o 'arbre', entenent la proximitat com una similitud calculada com a distància euclidiana o similitud del cosinus, per exemple.
- **Codificació de relacions lingüístiques entre paraules:** una característica si més no sorprenent d'aquestes transformacions és la codificació d'algunes relacions entre paraules com a transformacions lineals. Per exemple, la transformació lineal entre 'home' i 'dona' és similar a la necessària entre 'rei' i 'reina', 'oncle' i 'tia' o 'actor' i 'actriu', generalitzant per tant aquesta transformació com una transformació de gènere. Així, això habilita la possibilitat de fer operacions amb resultats aproximats com $\vec{\text{rei}} - \vec{\text{home}} + \vec{\text{dona}} \approx \vec{\text{reina}}$ o com $\vec{\text{París}} - \vec{\text{França}} + \vec{\text{Alemanya}} \approx \vec{\text{Berlín}}$

Les limitacions d'aquests mètodes procedeixen de la mateixa naturalesa del seu disseny. En estar entrenats per codificar el significat de les paraules aïllades, no codifiquen el context. Una possibilitat per integrar el context, entorn de les xarxes neuronals, va passar per utilitzar els *embeddings* com a transfor-

macions prèvies a l'entrada a les xarxes neuronals recurrents, amb l'objectiu que aquestes s'encarreguessin de codificar el context com a mitjà de resolució de problemes de processament natural concrets.

Els transformadors suposen un pas més enllà. L'objectiu d'aquests implica realitzar un aprenentatge d'inici a fi. Les transformacions de l'*embedding* passen a formar part de la mateixa xarxa i són apreses en temps d'entrenament, quedant integrades com a part de la xarxa global com els paràmetres d'autoatenció de la primera capa.

2.4.2. Inicis dels transformadors per a NLP

Les publicacions inicials que van marcar l'evolució posterior dels transformadors se circumscriuen en l'àmbit del processament natural. Presentem a continuació, en ordre cronològic, un breu resum d'aquelles que considerem més rellevants per a la definició d'aquest paradigma de disseny, així com les aportacions més significatives de cadascuna d'aquestes publicacions. Una introducció de l'arquitectura pot veure's a Vaswani i altres (2017):

- **ULMFiT** (Universal Language Model Fine-tuning for Text Classification). Els autors presenten una solució per resoldre qualsevol tasca de NLP mitjançant aprenentatge per transferència. Va introduir la idea de *general-domain*, aprenentatge no supervisat seguit de *fine-tuning*. Van introduir també altres tècniques àmpliament utilitzades posteriorment en diversos treballs com «Slanted triangular learning rate schedules (akas warm-up)» (Howard y Ruder, 2018).
- **GPT** (Radford i altres, 2018) i **GPT-2** (Radford i altres, 2019) (Generative Pre-Trained Transformers). Es presenta una arquitectura formada únicament per descodificadors. La major contribució d'aquestes publicacions és que van demostrar el valor d'entrenar models de mida enorme per a la predicció de la paraula següent que es presenta en un text.
- **BERT** (Devlin i altres, 2018) (Bidirectional Encoder Representations from Transformers.) Els autors utilitzen únicament codificadors per preentrenar representacions bidireccionals de text no etiquetat. Aquests models són afinats posteriorment amb un segon entrenament, afegint únicament una capa de sortida addicional, per resoldre aplicacions específiques. Aquest model va suposar un abans i un després en la resolució de problemes de NLP.
- **RoBERTa** (Liu i altres, 2019) (Robustly Optimized BERT Approach). Els autors proposen una arquitectura idèntica a BERT introduint una sèrie de millores del procediment d'entrenament. Bàsicament, és un BERT amb un entrenament millorat.

- **T5** (Raffel i altres, 2020) (Text-to-Text Transfer Transformer). Les principals contribucions d'aquest treball són, d'una banda, la reformulació de la manera de reportar els resultats substituint l'enfocament numèric per un enfocament textual i, de l'altra, l'estudi sistemàtic de la contribució de la funció objectiu, arquitectures, conjunts de dades, *transfer approaches* i altres factors en els resultats obtinguts en un conjunt de tasques canòniques de NLP.

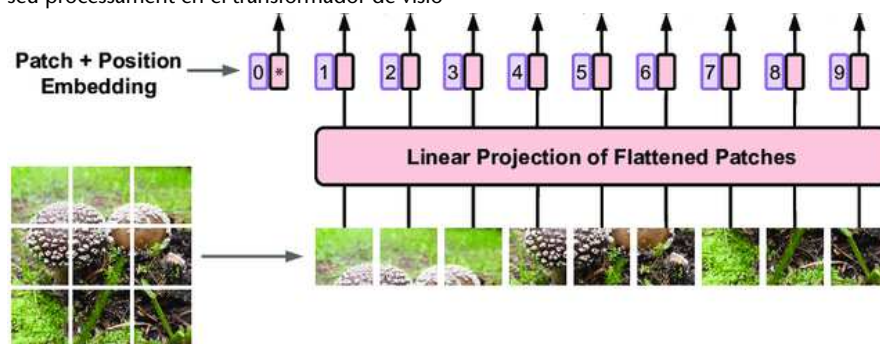
2.5. Transformadors per a Visió (ViT)

Als subapartats anteriors hem estudiat els transformadors en la seva aplicació de referència, el processament del llenguatge natural. Després dels prometedors resultats obtinguts en aquest camp, alguns autors van començar a considerar la possibilitat d'utilitzar aquesta arquitectura en altres camps com la visió.

Des de la seva introducció, es va demostrar que el rendiment dels models ViT és comparable al d'arquitectures especialitzades com són les xarxes convolucionals per a aplicacions típiques de visió per computador, com la classificació d'imatges (Dosovitskiy i altres, 2020), detecció d'objectes (Zhu i altres, 2020), segmentació semàntica (Zheng i altres, 2021b), coloració de la imatge (Kumar i altres, 2021), visió de baix nivell (Chen i altres, 2021a) i comprensió de vídeo (Arnab i altres, 2021), per citar-ne algunes. A més, recents recerques indiquen que els errors de predicció de ViT són més consistents amb els dels humans que les xarxes convolucionals (Tuli i altres, 2021).

El funcionament dels ViT és similar a un transformador convencional. L'única diferència essencial radica en la naturalesa del senyal d'entrada, que en comptes de ser una seqüència de paraules, és una imatge. La problemàtica radica, per tant, en com tractar l'entrada perquè sigui gestionable pel model; això és, adaptar el *tokenitzador* per treballar amb imatges. Una de les primeres propostes fetes a la bibliografia, amb molt bons resultats, consisteix a particionar la imatge en parts disjunctes més petites i alimentar-les a la xarxa de manera seqüencial. Cada part té les mateixes dimensions (16x16, per exemple, en Dosovitskiy i altres (2020)) i actua de manera equivalent al que seria la codificació d'un element en una seqüència. A aquest vector s'hi afegeix un *embedding* per codificar la posició que ocupa en la imatge, a l'estil del que es fa amb les seqüències de text per situar cada element.

Figura 14. Diagrama del procés de partició, seqüenciació i *embedding* d'una imatge per al seu processament en el transformador de visió



Font: Adaptat de Picek i altres (2022)

A Liu i altres (2021) es pot trobar un exhaustiu compendi de totes les referències existents fins al moment relatives a transformadors per a visió. Si bé l'objectiu principal de l'arquitectura dels transformadors és aconseguir un tractament generalista per a dades heterogènies, no totes les publicacions existents van en aquest sentit. Així, podem trobar articles que modifiquen els sistemes d'atenció per aconseguir les propietats de localitat semblants a les de les xarxes convolucionals mitjançant l'ús d'atenció local o jeràrquica, conceptes que descriurem a l'apartat dedicat als transformadors multimodals. Unes altres utilitzen com a entrada del transformador atributs representatius de la imatge procedents de xarxes convolucionals preentrenades. Encara que s'allunya de l'objectiu d'aquest mòdul presentar la miríada de variacions existents publicades que estan allunyades de l'esperit generalista de l'arquitectura, farem una presentació general de les propostes realitzades fins al moment sobre aquest tema. Cal tenir en compte que els transformadors, en general, i per a visió en particular, són una arquitectura novella, per tant no madura, que segur que estarà subjecta a grans innovacions futures. Independentment de l'interès que pugui suscitar una aplicació particular, l'objectiu d'aquest mòdul està destinat a explicar aquelles arquitectures més generalistes que funcionin bé en un ampli ventall de circumstàncies. Als subapartats que segueixen intentarem presentar aquelles arquitectures que compleixen aquesta comesa per a les aplicacions de visió en els camps de classificació, detecció i segmentació. Per a aplicacions més específiques es pot consultar l'extensa bibliografia.

2.5.1. Classificació

Fins al moment es pot dir que hi ha fins a quatre formes diferents d'abordar la contribució dels transformadors en la resolució de problemes de classificació, i que enumerem a continuació:

1) Adaptació directa de l'arquitectura original (Dosovitskiy i altres, 2020). La imatge original es disposa com un conjunt de parts més petites no superposades (*patches*), que són tractades com a elements d'una seqüència.

2) Transformador d'atributs procedents d'una xarxa convolucional (Wu i altres, 2020). Els atributs extrets d'una capa intermèdia d'una xarxa CNN preentrenada són alimentats a un transformador per millorar les seves capacitats de representació. L'avantatge d'aquest enfocament és que els elements de la seqüència inicial ja separen i codifiquen la informació, que és combinada i perfeccionada pel transformador. D'aquesta manera s'aprofiten els priors estructurals de la CNN per facilitar el treball posterior.

3) Destil·lació del coneixement d'una convolucional preentrenada (Touvron i altres, 2021). Una xarxa CNN actua com a mestra d'una xarxa basada en transformadors. El procés d'aprenentatge aconsegueix transferir els biaixos inductius a la xarxa aprenenta mitjançant destil·lació del coneixement (Hinton i altres, 2015). Sorprenentment, la xarxa aprenenta basada en transformadors acaba obtenint millors resultats predictius que els de la xarxa convolucional de la qual deriva.

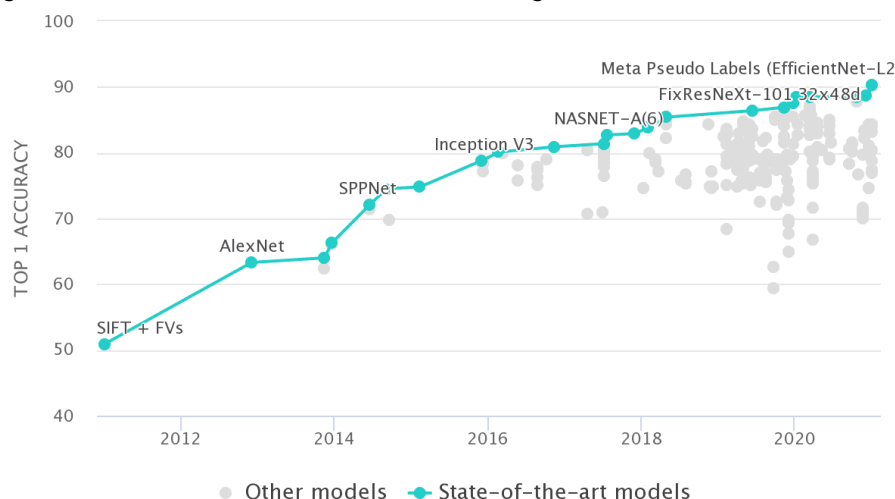
4) Transformador amb atenció localitzada (Liu i altres, 2021) i **transformadors jeràrquics**. És un transformador amb mòduls d'atenció adaptats per funcionar de forma localitzada. A partir de la partició de la imatge d'entrada en elements més petits que en ViT original (Dosovitskiy i altres, 2020) (4x4 en comptes de 16x16), aquests s'alimenten a matrius d'autoatenció que combinen únicament les particions més pròximes. Una vegada calculada l'atenció, es recombina amb les atencions circumdants mitjançant una capa de fusió. Les capes següents apliquen el mateix model d'atenció localitzada desplaçant les particions combinades. D'aquesta forma s'aconsegueix combinar la informació amb les zones adjacents. A Yuan i altres (2021) i a Wang i altres (2021) s'utilitzen aproximacions similars basades en un tractament jeràrquic on els mòduls d'atenció són modificats per integrar una progressiva reducció de l'espai de càlcul.

Destil·lació del coneixement

Conegut en anglès com a *Knowledge Distillation*, és el procés de transferència del coneixement d'una xarxa origen a una altra de destinació sense la necessitat de dades externes. A l'efecte d'aprenentatge, la xarxa d'origen actua de mestre, mentre que la de destinació actua com a alumne. Habitualment, la xarxa d'origen és més gran que la de destinació i ha estat entrenada prèviament a partir d'un conjunt de dades que no intervé en el procés.

A la figura 15 es mostra l'evolució temporal de l'estat de l'art per a classificació d'imatges (Benchmark Imagenet). Es pot observar com a mitjan 2022 els models amb millor rendiment estaven basats en transformadors (ViT-G/14, 1843M de paràmetres, 91.0% (Ding i altres, 2022)), seguits molt de prop d'un basat en xarxes convolucionals (EfficientNet-L2, 480M de paràmetres, 90.2% (Pham i altres, 2021)). Per fer comparatius els resultats, busquem un transformador amb un nombre similar de paràmetres. Trobem que DaViT-H, amb 362M aconsegueix també un 90.2% (Ding i altres, 2022). A partir d'aquests resultats es pot concloure que l'arquitectura dels transformadors, malgrat no disposar de priors estructurals, aconsegueix obtenir resultats de classificació parells i fins i tot millors de rendiment respecte a les CNN més competitives, amb una complexitat computacional similar. En contrapartida cal indicar que per aconseguir aquests resultats cal sotmetre les xarxes de transformadors a processos d'optimització més prolongats i amb conjunts de dades grans. Una alternativa quan no es disposa de suficients dades etiquetades és utilitzar una etapa prèvia d'aprenentatge semisupervisat (Weng i altres, 2021).

Figura 15. Estat de l'art dels models de classificació d'imatge



Font: ima

2.5.2. Detecció

Quan fem referència a l'ús de transformadors per a la detecció en imatges, hem de fer una distinció d'inici entre les aplicacions que utilitzen un enfocament de transformador d'extrem a extrem, d'aquelles que el fan servir únicament com a *backbone* per a l'extracció d'atributs, però utilitzen un bloc final de detecció convencional (per exemple, **RetinaNet** (Lin i altres, 2017b) o **Mask R-CNN** (He i altres, 2017)).

Els *backbone* esmentats venen a ser transformadors preentrenats en tasques de classificació que s'utilitzen com a extractor d'atributs per alimentar un capçal de detecció.

Pel que fa als transformadors d'extrem a extrem, en el moment de la redacció d'aquest apartat hi ha dos paradigmes diferents per abordar el problema de detecció: un seria la família de models derivats de l'arquitectura **DETR** (Carion i altres, 2020) i l'altre basat en models derivats de **Pix2Seq** (Chen i altres, 2021b). Estudiem a continuació les característiques definitòries de cadascun d'ells.

DETR: DEtection with TRansformer

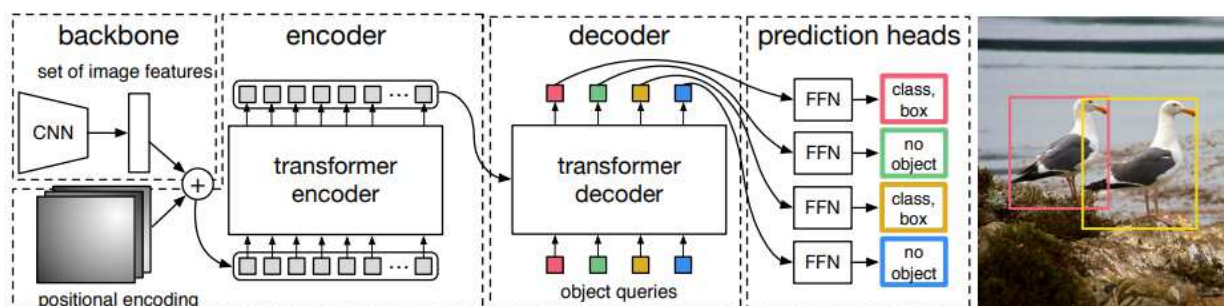
A la figura 16 es mostra esquemàticament l'arquitectura de detecció **DETR**. La imatge d'entrada es partitona, seqüència i alimenta un transformador seqüència a seqüència. El *tokenitzador* està format per una xarxa convolucional que extreu els atributs de la imatge original per posteriorment entrar en el transformador codificador. L'originalitat d'aquesta proposta ve en la part de descodificació. Aquesta, d'una banda, rep les dades procedents de la codificació i, de l'altra, una seqüència d'entrada. Aquesta representa la proposta

Bibliografia recomanada

Per a un compendi dels models que utilitzen aquest tipus d'enfocament es pot consultar Liu i altres (2021).

d'objectes a detectar. Al seu pas per la xarxa de descodificació es combina amb la informació de la imatge, donant com a resultat la seqüència de sortida. Si la xarxa ha complert la seva funció, la seqüència de sortida continuarà la representació interna de la detecció dels objectes presents en la imatge. La seqüència d'elements de sortida té la mateixa cardinalitat que la d'entrada i alimenta una xarxa neuronal independent per a cada element. D'aquesta manera, es deriva la classe i el quadre delimitador dels elements detectats. La mida de la seqüència del descodificador determina el nombre màxim d'elements que és possible detectar. A l'article original, el nombre màxim d'objectes és de 100. El model preveu una classe especial per indicar la no-presència d'objecte per a cada posició. A **DETR** la seqüència d'entrada està formada per nombres aleatoris que serveixen com a prior per a la detecció dels objectes. Malgrat la naturalesa aleatòria del prior de **DETR** els resultats obtinguts són molt bons. Treballs publicats posteriorment han intentat millorar les capacitats predictives de la xarxa afinant els priors de la seqüència d'entrada (**SMCA** (Gao i altres, 2021), **Conditional DETR** (Meng i altres, 2021), **Anchor DETR** (Wang i altres, 2022), **DAB-DETR** (Liu i altres, 2022), **Efficient DETR** (Yao i altres, 2021), **Dynamic DETR** (Dai i altres, 2021)).

Figura 16. Arquitectura del transformador **DETR** especialitzat en detecció



Font: Carion i altres (2020)

Un punt clau del disseny d'aquesta arquitectura és el disseny d'una funció de pèrdua que permeti l'optimització eficient de la xarxa durant l'aprenentatge supervisat, és a dir, la comparació dels valors predits amb els reals. Per comprendre la dificultat del problema, podem veure l'exemple indicat a la taula 3. Observem com les classes predites i reals no necessàriament es trobaran en la mateixa posició. A més, l'ordenament de les prediccions tampoc garanteix la coincidència, perquè serà freqüent el cas en què el model no sigui capaç de predir tots els elements o que ho faci de manera errònia. Aquest problema es resol calculant la funció de pèrdua per a totes les agrupacions possibles entre valors veritables i predits, triant aquella combinació on la pèrdua és menor (Carion i altres, 2020), (Stewart i altres, 2016).

Taula 3. Exemple de valor real i predit en una xarxa de detecció

Real			Predit		
Classe	Des de	Fins a	Classe	Des de	Fins a
got	(100, 200)	(200, 300)	sense objecte	(20, 40)	(10,0)
ampolla	(300, 200)	(400, 300)	got	(98, 198)	(210,310)
bol	(100, 200)	(200, 300)	sense objecte	(20, 40)	(10,0)
portàtil	(100, 200)	(200, 300)	sense objecte	(20, 40)	(10,0)
ratolí	(100, 200)	(200, 300)	sense objecte	(20, 40)	(10,0)
cadira	(500, 400)	(700, 500)	bossa	(460, 560)	(510, 599)
bossa	(450, 550)	(500, 600)	sense objecte	(20, 40)	(10,0)
sense objecte	(0, 0)	(0, 0)	sense objecte	(20, 40)	(10,0)
sense objecte	(0, 0)	(0, 0)	ampolla	(20, 40)	(10,0)
sense objecte	(0, 0)	(0, 0)	sense objecte	(20, 40)	(10,0)

La funció de pèrdua calculada amb l'aparellament bipartit és la que es mostra a l'equació 6.

$$\hat{\sigma} = \underset{\sigma \in \mathcal{G}_N}{\operatorname{argmin}} \sum_i^N \mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)}), \quad (6)$$

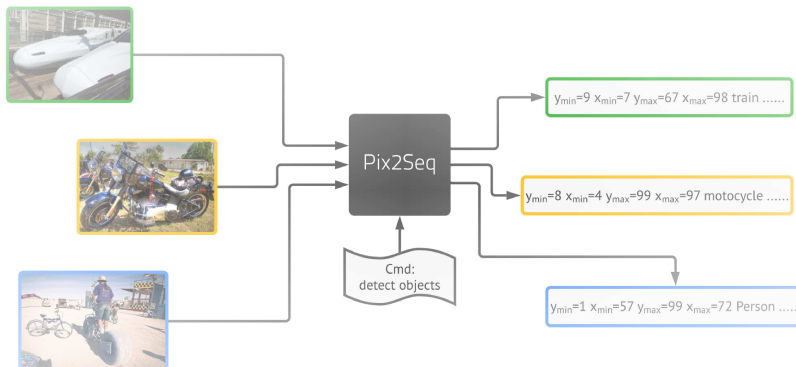
$$\mathcal{L}_{\text{match}}(y_i, \hat{y}_{\sigma(i)}) = -\mathbb{1}_{\{c_i \neq \emptyset\}} \hat{p}_{\sigma(i)}(c_i) + \mathbb{1}_{\{c_i \neq \emptyset\}} \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\sigma(i)})$$

Una vegada fet l'aparellament òptim, per dur a terme l'optimització de la xarxa s'utilitza la funció de pèrdua hongaresa (equació 7).

$$\mathcal{L}_H(y_i, \hat{y}_{\sigma(i)}) = \sum_{i=1}^N [-\log \hat{p}_{\hat{\sigma}_i}(c_i) + \mathbb{1}_{\{c_i \neq \emptyset\}} \mathcal{L}_{\text{box}}(b_i, \hat{b}_{\sigma(i)})] \quad (7)$$

Pix2Seq

A la figura 17 es mostra un esquema del funcionament de **Pix2Seq**. Utilitza un nou paradigma consistent en un mapatge directe entre la imatge a tractar i la sortida de seqüència de text, on es codifiquen classe i quadre delimitador de tots els elements detectats. És una solució elegant amb resultats de rendiment similars als obtinguts per **DETR** i fins i tot en algunes circumstàncies millors (per exemple, en la detecció d'objectes petits).

Figura 17. Tipologia d'E/S del transformador **Pix2Seq** especialitzat en detecció

Font: Carion i altres (2020)

Utilitza una xarxa de transformadors formada per codificadors i descodificadors resolent de forma autoregressiva. La part de codificació pot ser o bé un transformador o una xarxa convolucional. El codificador extreu els atributs d'interès de la imatge per alimentar-los a la xarxa de descodificadors, que els converteixen juntament amb els comandaments procedents de la seqüència d'entrada en una sortida textual. Es genera un element cada vegada, condicionat als atributs subministrats pel codificador i als elements de la seqüència generats fins al moment.

La naturalesa textual de la sortida simplifica els càlculs requerits de la funció de pèrdua respecte a DETR. El format esperat de la sortida és un conjunt d'elements de la forma $[x_{min}], [y_{min}], [x_{max}], [y_{max}], [clase], \dots, [EOS]$, on EOS seria l'indicador de final de seqüència que apareix després de la sortida de tots els «caixetins» delimitadors presents a la imatge. Durant l'entrenament, l'ordenació dels elements en la seqüència de predicció es fa de manera aleatòria. S'optimitza mitjançant la funció de pèrdua típica de *log-likelihood* utilitzada pels transformadors autoregressius de NLP (equació 8).

$$\max \sum_{j=1}^L \omega_j \log P(\tilde{\mathbf{i}}_j | \mathbf{x}, \mathbf{i}_{1:j-1}) \quad (8)$$

on $\mathbf{x}$ és una imatge determinada, $\mathbf{y}$ i $\tilde{\mathbf{y}}$ les seqüències d'entrada i objectiu, respectivament, associades amb $\mathbf{x}$ i L la longitud de la seqüència. ω_j és un pes preestablert per a l'element j de la seqüència. En el cas de Pix2Seq, $\omega_j = 1, \forall j$.

2.5.3. Segmentació

Segmentació és el terme genèric que s'utilitza per referir-se en realitat a tres problemes diferents: **segmentació semàntica**, **segmentació d'instàncies** i **segmentació panòptica**. La **segmentació semàntica** és el problema consistent a assignar a cada píxel d'una imatge una classe. Aquest tipus de segmentació és el més senzill, ja que no requereix la separació entre objectes diferents pertanyents a una mateixa classe. A la **segmentació panòptica** i a la **segmentació d'instàncies** l'objectiu és més complex, perquè no únicament pretenen separar classes, sinó també objectes. La diferència entre ambdues és que la panòptica identifica cada píxel amb una única classe, mentre que la d'instàncies permet que un píxel estigui associat a més d'una classe, tenint en compte la superposició d'objectes.

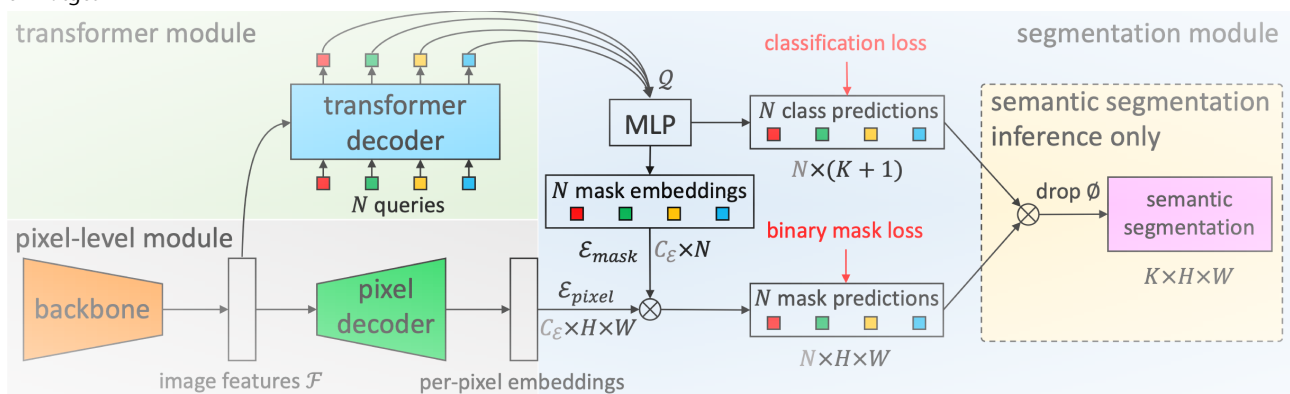
Les propostes per resoldre els problemes de segmentació utilitzant transformadors han estat i estan sotmeses a una profunda evolució. Podem distingir entre dos enfocaments: d'una banda, la segmentació directa a partir de la

informació dels píxels de la imatge i, de l'altra, la construcció de representacions internes basades en objectes, per inferir posteriorment les màscares de segmentació.

Les propostes inicials basades en el mètode directe són una extensió del ViT per a la resolució de tasques de segmentació (vegeu **SETR** (Zheng i altres, 2021b)). Aquesta primera aproximació va servir per demostrar la viabilitat de l'arquitectura per a la resolució d'aquesta mena de problemes, encara que amb uns costos computacionals elevats. Posteriorment, es van presentar propostes basades en la representació interna d'objectes amb arquitectures derivades de l'arquitectura de detecció **DETR** presentada al subapartat anterior. Les primeres propostes requerien classe, quadre delimitador i màscara. Les posteriors són capaces de derivar màscara i classe directament sense la necessitat de quadre delimitador. Presentarem en aquest subapartat l'arquitectura **Mask2Former** (Cheng i altres, 2022), com un exemple d'una arquitectura madura que permet resoldre amb una única arquitectura les tres tasques de segmentació: semàntica, d'instàncies i panòptica.

A la figura 18 es mostra l'esquema representatiu del transformador de segmentació **MaskFormer** (vàlid també per a **Mask2Former**).

Figura 18. Transformador MaskFormer. Arquitectura especialitzada en segmentació d'imatges



Font: Cheng i altres (2021)

Podem observar que consta d'un *backbone*, que pot ser tant una xarxa convolucional com un ViT, del qual s'extreuen els atributs representatius de la imatge. Aquests s'alimenten a un descodificador de píxels i a un transformador descodificador. El descodificador de píxels és una FPN (Feature Pyramid Network) (Lin i altres, 2017a), però podria ser qualsevol altra arquitectura que sigui capaç de transformar atributs a imatge (un exemple alternatiu podria ser una O-NET (Ronneberger i altres, 2015)). El transformador descodificador, a l'estil del **DETR**, genera una seqüència de sortida a partir d'una seqüència d'entrada utilitzant la representació interna dels objectes presents a la imatge.

procedent del *backbone*. A partir d'aquesta, un MLP que genera la predicció de classes per a cada objecte, així com una representació interna de la màscara, que, combinada amb la informació procedent de la sortida del descodificador de píxels, dona com a resultat les prediccions de les màscares per a cadascun dels objectes. Igual que amb DETR, hi ha una classe especial per identificar aquelles classes en les quals no s'ha detectat objecte i per a les quals, per tant, la segmentació reportada no ha de tenir-se en compte. El nombre màxim de màscares que la xarxa és capaç de detectar és un valor de disseny de l'arquitectura i està limitat per la mida de la seqüència d'objectes.

2.6. Transformadors per a àudio

Els transformadors també s'estan convertint en l'estàndard *de facto* per al tractament de senyals d'àudio, espai fins al moment ocupat per les xarxes convolucionals i anteriorment per les xarxes recurrents. Les possibilitats en aquest camp són molt àmplies i diverses. L'arquitectura utilitzada és la mateixa que en les aplicacions anteriorment presentades per a una altra mena de senyals, i el fet distintiu és el *tokenitzador* utilitzat per transformar el senyal d'entrada en una representació interna manejable per la xarxa.

2.6.1. Senyals d'àudio, tractament i tokenització

Les entrades de text i imatge són el tipus de dades més intuïtives de representar. Per al cas del so, sembla més complex el mode d'expressió numèrica. Farem una explicació de la naturalesa dels senyals d'àudio per intentar comprendre millor com es realitza la preparació per a la seva utilització en aplicacions de predicció automàtica.

Un senyal ve a ser l'evolució temporal d'una variable física que pot ser mesurada. En el cas de l'àudio, estariem referint-nos a variacions de la pressió de l'aire en un determinat punt. Per tractar digitalment aquests senyals, primer cal emmagatzemar-los. Això es fa inicialment capturant aquesta informació temporal realitzant un mostreig, això és, registrant el valor de la propietat física amb una freqüència preestablerta. Aquesta freqüència de mostreig pot variar, però un valor habitual per a so pot ser, per exemple, 44.1 kHz, això és, 44100 mostres per segon. Una vegada capturada aquesta informació, es disposa de la representació digital del senyal d'àudio i està en condicions de ser tractat digitalment.

De l'anàlisi matemàtica coneixem el teorema de Fourier, que ve a dir que tot senyal periòdic definit en un interval temporal finit, independentment de la seva forma, pot ser aproximat per la suma de senyals senoïdals. Cadascun d'aquests senyals constituents està definit per únicament dues variables: amplitud i freqüència. Aquestes variables poden ser mostrades gràficament en el

que es denomina un espectre de freqüències. Així, és possible representar el mateix senyal de dues formes diferents: atès el domini temporal o el domini de freqüència. A la transformació entre un domini i l'altre es denomina transformada de Fourier (Bracewell y Bracewell, 1986). L'algorisme FFT (Fast Fourier Transform) (Nussbaumer, 1981) permet dur a terme de manera eficient aquesta transformació en un entorn digital.

El que s'ha presentat fins ara aplica a senyals periòdics, però la parla o la música no són periòdiques, què ocorre llavors en aquests casos? La solució és definir una longitud de finestra en la qual es tracta el senyal com a periòdic i per al qual es calcula el seu espectre. Duent a terme aquest càlcul per a finestres consecutives superposades, obtenim una representació en domini de freqüència del senyal no periòdic. A aquest gràfic se'l denomina espectrograma. L'espectrograma és una representació gràfica de la freqüència respecte del temps. Cada punt representa l'amplitud en color en decibels. Els decibels venen a representar l'amplitud en escala logarítmica. S'utilitza aquesta escala pel fet que la percepció dels canvis de so en humans està més alineada amb aquest ordre d'escala.

Igual que passa amb l'amplitud, els humans no percebem les freqüències de manera lineal. La nostra oïda està més ben adaptada a percebre les diferències per a baixes freqüències (sons greus). Una persona sana és capaç de distingir sense problemes la diferència entre un senyal a 500 Hz d'un de 1000 Hz, però li serà molt més difícil fer-ho entre un de 10000 Hz i un de 10500 Hz, a pesar que la diferència entre tots dos és la mateixa. El 1937, es va proposar l'escala de MEL (Stevens i altres, 1937), que pretenia linealitzar la percepció de la diferència de freqüències introduint una nova unitat denominada *pitch* de MEL. El gràfic resultant de representar l'amplitud en decibels en un gràfic bidimensional de *pitch* vs. temps és més representatiu, linealment, de la percepció del sentit de l'oïda en l'ésser humà. A aquest gràfic se'l denomina espectrograma de MEL. Sol ser l'espectrograma més habitual en els models d'aprenentatge automàtic. La linealitat del gràfic amb la percepció auditiva humana emfatitza la informació rellevant i facilita el processament posterior del senyal.

Arribats a aquest punt podem entendre que una de les representacions més habituals dels senyals d'àudio per a l'aprenentatge automàtic sigui l'espectrograma de MEL. En ser una matriu de dues dimensions que sol tenir una mida més gran del que és gestionable directament pel transformador, un *tokenitzador* habitual (d'altres també són possibles) consisteix a aplicar convolucions que redueixin la seva mida a l'espai intern de l'arquitectura.

2.6.2. Exemples de transformadors d'àudio

A continuació es presenta una mostra de les moltes altres aplicacions d'àudio que utilitzen transformadors:

- **Whisper** (Radford i altres, 2022). És un transformador seqüència a seqüència multilinguatge que, a partir d'una font d'àudio que contingui una conversa, retorna la seqüència de text associada.
- **Wav2Vec 2.0** (Baevski i altres, 2020). És un transformador amb una finalitat similar a BERT, però que en comptes d'utilitzar seqüències de text treballa directament amb seqüències d'àudio. Utilitza una xarxa convolucional com a codificador d'atributs que posteriorment s'alimenten al transformador. Realitza un aprenentatge no supervisat per obtenir representacions internes de seqüències d'àudio. El model pot ser posteriorment afinat amb relativament poques dades per resoldre problemes específics.
- **SepFormer** (Subakan i altres, 2021). És una xarxa de transformadors especialitzada en la separació de veus d'una font d'àudio.

2.7. Transformadors multimodals

Fins ara hem vist la potencialitat dels transformadors per a l'aprenentatge automàtic utilitzant dades d'una única naturalesa. A causa de les seves capacitats generalistes per resoldre problemes d'àrees fins ara inconnexes, és lògic hipotetitzar que aquesta arquitectura potencialment pot ser capaç de gestionar problemes que requereixin entrades de diferent naturalesa al mateix temps. Un exemple clar d'agent d'aquest tipus són els animals i les persones, que integren informació procedent de fins a cinc sentits diferents.

Les arquitectures multimodals són aquelles capaces de gestionar entrades de diferent naturalesa, per exemple, imatge i text.

Des del punt de vista de la geometria topològica, l'espai generat per l'arquitectura dels transformadors és equivalent a la d'un graf completament connectat (Bronstein i altres, 2021). Això no és així en altres arquitectures, on les interconnexions del graf estan limitades al seu espai més restringit. Els transformadors, per tant, disposen d'un espai de modelatge més general i flexible (Xu i altres, 2022). Això representa un avantatge quan es requereix combinar entrades de diferent naturalesa. Els priors estructurals que poden ser útils per facilitar l'optimització per a un determinat tipus de dades poden ser un inconvenient quan s'utilitzen juntament amb altres entrades de diferent naturalesa.

A continuació es descriuen els punts diferencials que hi ha entre una arquitectura multimodal i la tradicional.

2.7.1. Entrades multimodals

La primera funció que realitza un transformador és la conversió de les dades d'entrada en un tensor de dimensió interna definida, que es manté constant en el seu avanç per les diferents capes. La *tokenització* transforma els elements d'entrada en un vector intermedi al qual posteriorment s'aplica una transformació matemàtica que, normalment, redueix les seves dimensions amb l'objectiu d'aconseguir les internes de procés. En entrades textuais és habitual utilitzar un *tokenitzador* que converteix cada element de la cadena d'entrada en un vector ortogonal de dimensió igual a la del diccionari. Aquesta operació és seguida d'una transformació lineal amb l'objectiu d'aconseguir la dimensió interna de treball. En el cas d'imatge, una estratègia consisteix a *tokenitzar* mitjançant la partició de la imatge en parts no superposades de mida predefinida, seguit d'una transformació lineal d'aquesta per aconseguir la dimensió de treball (Dosovitskiy i altres, 2020).

Un altre tipus d'estratègies de *tokenització* consisteix en l'ús de xarxes especialitzades per derivar atributs que són alimentats al transformador (Lu i altres, 2019). En el cas del vídeo, hi ha diverses estratègies: a Sun i altres (2019) la *tokenització* consisteix a seleccionar clips de durada determinada, a uns fps preestablerts, als quals seguidament se'ls passa a través d'una xarxa convolucional tridimensional per generar uns atributs que són posteriorment adaptats fins a aconseguir les dimensions internes de treball. Altres estratègies publicades inclouen la *tokenització* mitjançant selecció en els clips de punts tridimensionals d'interès, seguida per una transformació lineal (Akbari i altres, 2021); *tokenització* mitjançant la partició de les imatges integrants aplicada al conjunt del clip seguida d'una projecció lineal (Nagrani i altres, 2021), entre altres. En el cas de l'àudio, es poden utilitzar també diferents estratègies que depenen en part de l'objectiu que es pretén aconseguir. Una d'elles consisteix, com hem descrit anteriorment, a utilitzar l'espectrograma MEL (o altres derivats) seguit d'una projecció lineal o del càlcul d'atributs mitjançant una xarxa convolucional (Lin i altres, 2021), (Akbari i altres, 2021), (Nagrani i altres, 2021).

Lectura recomanada

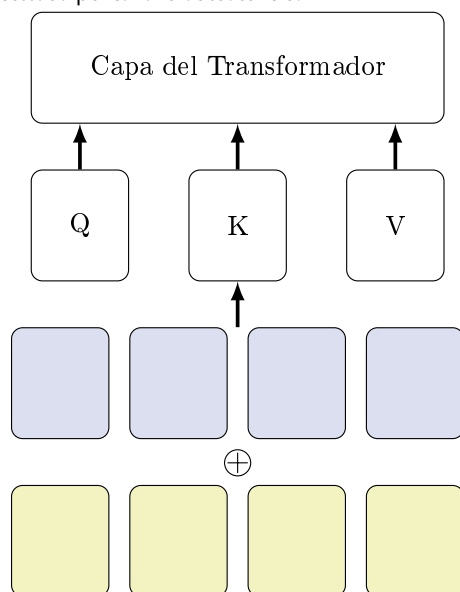
A Xu i altres (2022) es poden trobar exemples de mòduls preparatoris per a una altra mena de dades més específiques, com podrien ser esquemes de bases de dades SQL, núvols de punts tridimensionals, dades tabulars, poses humanes, registres electrònics de salut, conjunt d'ordres possibles a les quals respon un robot, entre molts d'altres.

2.7.2. Variants d'autoatenció en un context multimodal

Una vegada es disposa de la representació interna dels diferents senyals d'entrada, es planteja el dilema de com trobar la manera més efectiva de combinar-les en la capa d'autoatenció per obtenir els millors resultats. Malgrat que puguin tenir les mateixes dimensions, a causa de la seva diferent naturalesa, pot ser necessari adaptar les transformacions a cadascuna de les diferents formes. Les estratègies més habituals per calcular l'autoatenció en aquesta primera etapa són: la suma primerenca, la concatenació primerenca, la jeràrquica i la creuada.

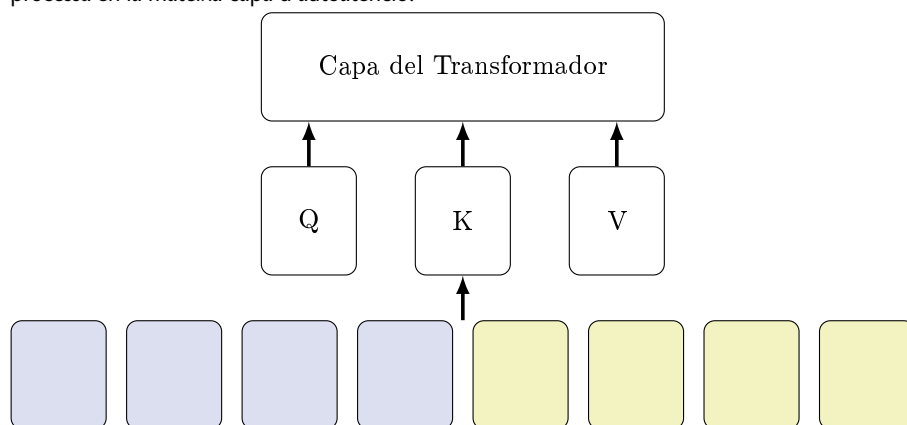
La **suma primerenca** (figura 19) consisteix en la suma dels vectors procedents de les diferents fonts com a pas previ a un únic sistema d'autoatenció comú.

Figura 19. Atenció per suma primerenca. La informació procedent de dos canals diferents se suma abans de ser processada pel canal d'autoatenció.



La **concatenació primerenca** (figura 20) genera un vector com a concatenació de les diferents maneres que s'alimenta al mòdul d'autoatenció.

Figura 20. Atenció per concatenació. La informació procedent de dos canals diferents es processa en la mateixa capa d'autoatenció.



L'**autoatenció jeràrquica** (figures 21 i 22) es realitza en dos passos, en un s'apliquen transformacions diferents a cada senyal i en l'altre una única que les combina.

Figura 21. Atenció jeràrquica multicanal a monocanal. La informació procedent de canals diferents es processa en una primera etapa en canals d'atenció independents per posteriorment processar-se en conjunt a l'etapa final.

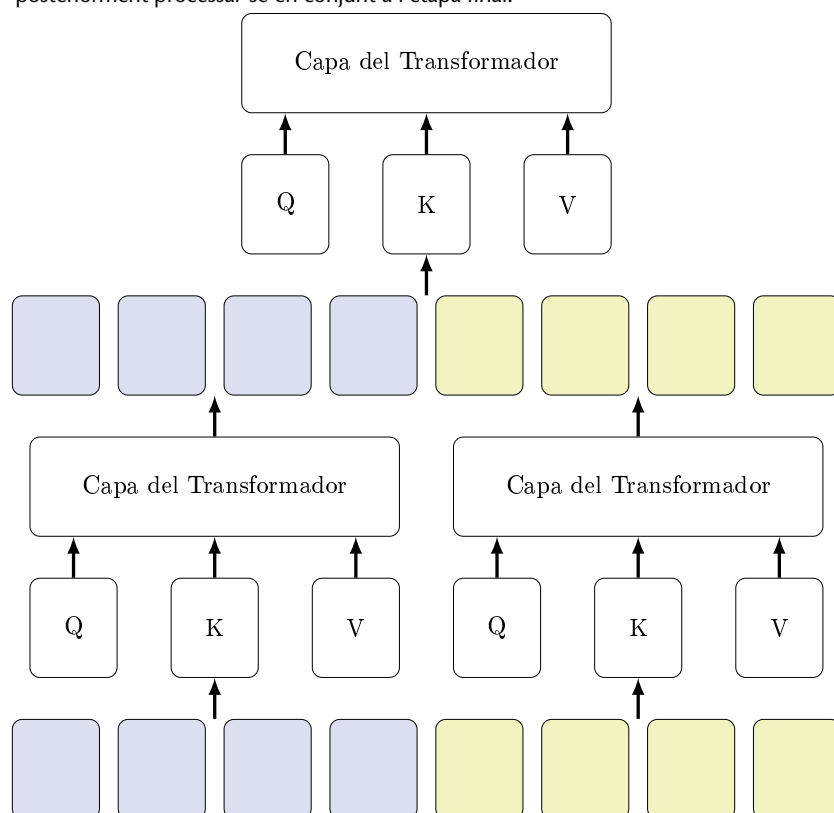
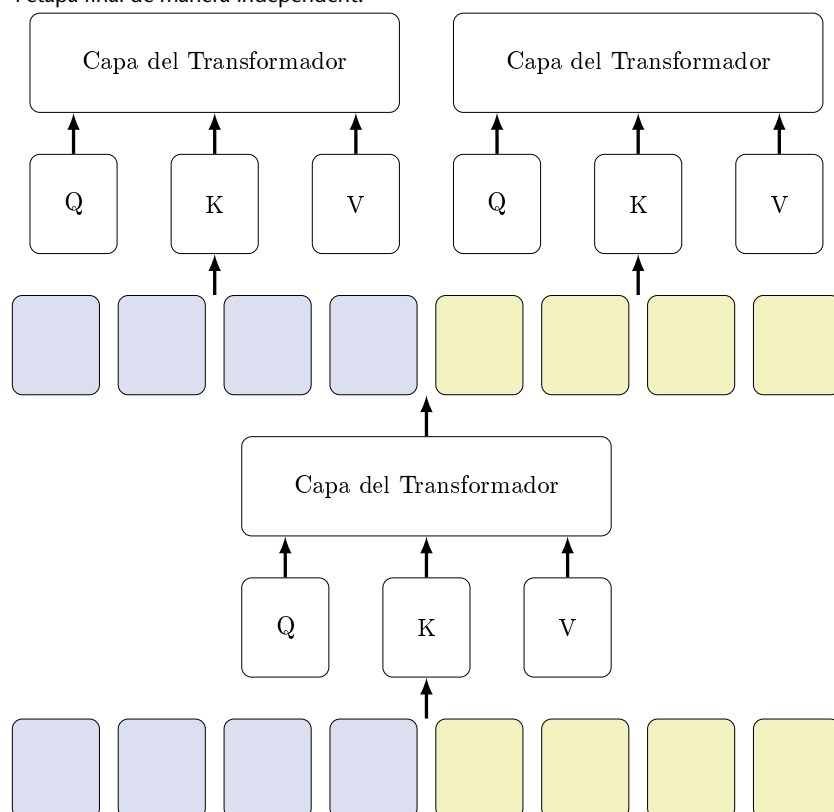
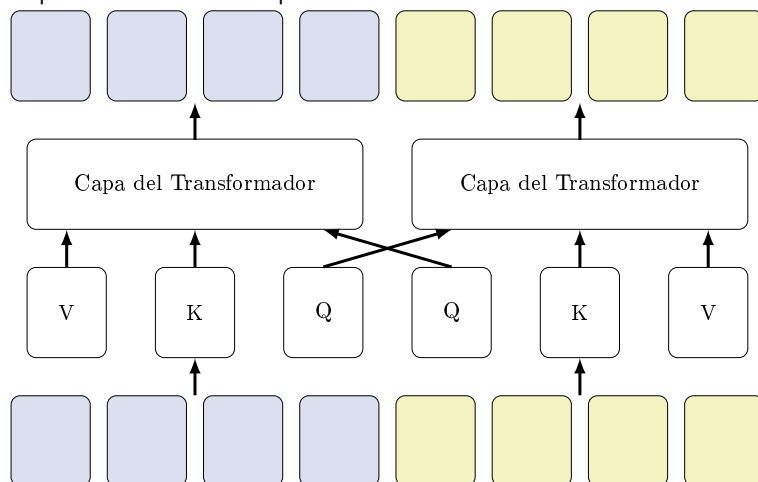


Figura 22. Atenció jeràrquica monocanal a multicanal. La informació procedent de canals diferents es processa en una primera etapa en conjunt per posteriorment processar-se en l'etapa final de manera independent.



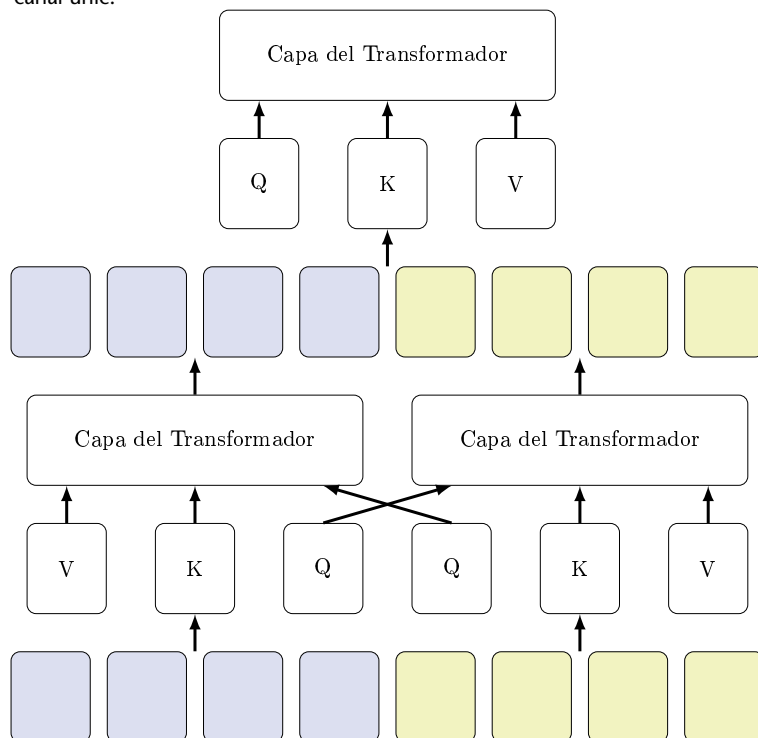
L'**autoatenció creuada** (figura 23) utilitza canals d'autoatenció independents on la matriu de Query dels canals actua sobre l'altre canal i no sobre el propi.

Figura 23. Atenció creuada. La informació procedent de dos canals diferents es processa en la mateixa capa d'autoatenció.



L'**autoatenció creuada seguida de concatenació** (figura 24) actua en dues fases, una primera en què s'aplica una autoatenció creuada i, posteriorment, s'aplica una autoatenció convencional sobre la concatenació dels vectors procedents de l'operació original.

Figura 24. Atenció creuada seguida de concatenació. La informació procedent de dos canals diferents es processa en la mateixa capa d'autoatenció i, posteriorment, és combinada en un canal únic.



Taula 4. Variants d'autoatenció d'interacció/fusió de canals multimodals. α i β són pesos.

Att: Atenció; $\mathcal{C}$: Concatenació; Tfs: Capes de transformador. $N_{(A)}$ i $N_{(B)}$ longituds de seqüència de dos canals A i B de diferent forma. $MHSA(\mathbf{Q}, \mathbf{K}, \mathbf{V})$ Funció d'autoatenció multicapçal

autoatenció	Formulació	Referències
Suma primerenca	$\mathbf{Z} \leftarrow Tf(\alpha \mathbf{Z}_{(A)} \oplus \beta \mathbf{Z}_{(B)})$	(Gavrilyuk i altres, 2020), (Xu y Zhu, 2021)
Concatenació primerenca	$\mathbf{Z} \leftarrow Tf(\mathcal{C}(\mathbf{Z}_{(A)}, \mathbf{Z}_{(B)}))$	(Sun i altres, 2019), (Guo i altres, 2020), (Shi i altres, 2022), (Zheng i altres, 2021a)
Jeràrquica multi $\rightarrow$ monocanal	$\mathbf{Z} \leftarrow Tf_3(\mathcal{C}(Tf_1(\mathbf{Z}_{(A)}), Tf_2(\mathbf{Z}_{(B)})))$	(Li i altres, 2021)
Jeràrquica mono $\rightarrow$ multicanal	$\begin{cases} \mathbf{Z}_{(A)} \leftarrow Tf_2(Tf_1(\mathcal{C}(\mathbf{Z}_{(A)}, \mathbf{Z}_{(B)}))) \\ \mathbf{Z}_{(B)} \leftarrow Tf_3(Tf_1(\mathcal{C}(\mathbf{Z}_{(A)}, \mathbf{Z}_{(B)}))) \end{cases}$	(Lin i altres, 2020)
Creuada	$\begin{cases} \mathbf{Z}_{(A)} \leftarrow MHSA(\mathbf{Q}_B, \mathbf{K}_A, \mathbf{V}_A) \\ \mathbf{Z}_{(B)} \leftarrow MHSA(\mathbf{Q}_A, \mathbf{K}_B, \mathbf{V}_B) \end{cases}$	(Lu i altres, 2019), (Yun i altres, 2021)
Creuada més concatenació	$\begin{cases} \mathbf{Z}_{(A)} \leftarrow MHSA(\mathbf{Q}_B, \mathbf{K}_A, \mathbf{V}_A) \\ \mathbf{Z}_{(B)} \leftarrow MHSA(\mathbf{Q}_A, \mathbf{K}_B, \mathbf{V}_B) \\ \mathbf{Z} \leftarrow Tf(\mathcal{C}(\mathbf{Z}_{(A)}, \mathbf{Z}_{(B)})) \end{cases}$	(Hasan i altres, 2021), (Zhan i altres, 2021), (Tsai i altres, 2019)

2.7.3. Arquitectures multimodals

L'objectiu d'aquestes estratègies és la consecució d'una representació interna que integri de la manera més adequada possible la informació rellevant procedent de cadascuna de les formes d'entrada. El sistema d'autoatenció triat determina les característiques de l'arquitectura. Així, parlem d'arquitectures de **canal únic**, **multicanal** o **híbrides**. Les arquitectures de **canal únic** són les derivades de l'ús dels sistemes d'autoatenció de **suma** i **concatenació primerenca** i disposen d'un canal de tractament únic de les dades. Les arquitectures **multicanal** són les derivades de l'ús dels sistemes d'autoatenció **creuada** i donen lloc a canals múltiples de tractament diferenciat de les dades. Finalment, les arquitectures **híbrides**, derivades de sistemes d'autoatenció **jeràrquica** i **creuada amb concatenació** combinen parts de les dues anteriors.

2.8. Aplicacions dels transformadors multimodals

En aquest subapartat podem veure alguns dels treballs publicats fins al moment que utilitzen transformadors multimodals.

- **MMBT** (Kiela i altres, 2019) és una arquitectura multimodal que integra imatge i text per obtenir una representació interna que pot ser utilitzada per fer tasques de classificació.
- **VideoBERT** (Sun i altres, 2019) és una arquitectura multimodal amb una finalitat similar a BERT però utilitzant vídeo com a font de dades.
- **Med-BERT** (Rasmy i altres, 2021) és una arquitectura multimodal que utilitza registres de salut estructurats per representar la salut de pacients i a partir d'aquesta poder predir malalties.

- **Gato** (Reed i altres, 2022) és una arquitectura multimodal amb un objecte generalista. A partir de dades procedents de diverses fonts (imatge, text i accions pròpies d'un robot) és capaç de resoldre fins a 604 tasques diferents, entre les quals es troben jugar a multitud de jocs d'Atari, fer de xatbot, comandar un braç articulat, etiquetar el contingut d'una imatge, entre moltes altres. Tot això utilitzant una única arquitectura basada en transformadors. És la primera aplicació generalista que integra imatge, processament del llenguatge natural, aprenentatge per reforç i control de robots en un únic agent.

Resum

En aquest mòdul hem presentat l'arquitectura de xarxa neuronal de transformadors, la qual té la capacitat de reemplaçar arquitectures especialitzades com xarxes convolucionals, recurrents i de grafs. Aquesta arquitectura no sols pot funcionar en el camp específic del processament del llenguatge natural, per al qual va ser dissenyada originalment, sinó que també ha demostrat funcionar al màxim nivell per al tractament d'una altra mena de senyals específics, així com per a la resolució de problemes que combinen dades de diferents fonts, la qual cosa la converteix en un agent generalista per a la resolució de problemes.

L'arquitectura dels transformadors ha demostrat funcionar molt bé en àrees com la visió per computador, el processament del llenguatge natural, el tractament de senyals d'àudio, la predicció de sèries temporals i l'aprenentatge per reforç, entre altres.

En el mòdul s'ha presentat una descripció rigorosa de l'algorisme de l'arquitectura i s'han mostrat aplicacions en diferents àrees, des de la seva aplicació d'origen en el processament del llenguatge natural, el processament d'imatges i sons i fins i tot el seu ús en aplicacions multimodals.

Conèixer els detalls d'implementació de l'arquitectura dels transformadors és de gran importància, ja que pot ser una solució general per a la majoria dels problemes de ciència de dades.

Glossari

atenció *f* mecanisme mitjançant el qual és possible emfatitzar la importància d'una part de la informació present en un vector multidimensional, representació interna d'un estat.

embedding *m* representació numèrica d'un *token*. Sol ser apresada mitjançant una transformació que pot ser lineal o no lineal a partir de la codificació del *token* en un *vector one-hot*.

NLP sigles en anglès de *Natural Language Processing*, en català, 'Processament del Llenguatge Natural'.

NLU sigles en anglès de *Natural Language Understanding*, en català, 'Comprensió del Llenguatge Natural'.

tokenització *f* subdivisió dels elements d'una seqüència d'entrada en subelements més petits. Per exemple, una frase en paraules, caràcters, agrupacions de caràcters comuns, etc. A cada element constituent se'n diu *token*.

transformador *m* arquitectura de xarxa neuronal que està formada per un conjunt de capes que integren en cadascuna d'elles mecanismes d'atenció.

vector one-hot *m* vector representatiu del valor d'un *token*. Cada *token* diferent s'associa a una posició diferent atenent la seva correspondència amb un diccionari. El *token* queda representat per un vector ortogonal que té un u en la posició que li correspon dins del diccionari i zeros en totes les altres.

Notació

La notació matemàtica d'aquest mòdul és la mateixa que la utilitzada a Phuong y Hutter (2022). A efectes pràctics la reproduïm a continuació.

Símbol	Tipus	Explicació
$[N]$	$:= \{1, \dots, N\}$	conjunt d'enters $1, 2, \dots, N - 1, N$
i, j	$\in \mathbb{N}$	índexs sencers genèrics
V	$\cong [N_V]$	vocabulari
N_V	$\in \mathbb{N}$	mida del vocabulari
V^*	$= \bigcup_{\ell=0}^{\infty} V^\ell$	conjunt de seqüències de <i>tokens</i> ; per exemple, paraules i documents
ℓ_{max}	$\in \mathbb{N}$	longitud màxima de la seqüència de <i>tokens</i>
ℓ	$\in [\ell_{max}]$	longitud de la seqüència de <i>tokens</i>
t	$\in [\ell]$	índex del token dins d'una seqüència
$d \dots$	$\in \mathbb{N}$	dimensió de diversos vectors
$\mathbf{x}$	$\equiv x[1 : \ell]$	$\equiv x[1]x[2] \dots x[\ell] \in V^\ell$ seqüència de <i>tokens</i> primària
$\mathbf{z}$	$\equiv z[1 : \ell]$	$\equiv z[1]z[2] \dots z[\ell] \in V^\ell$ seqüència de <i>tokens</i> de context
$M[i, j]$	$\in \mathbb{R}$	entrada $M_{i,j}$ de la matriu $M \in \mathbb{R}^{d \times d'}$
$M[i, :] \equiv M[i]$	$\in \mathbb{R}^{d'}$	fila i de la matriu $M \in \mathbb{R}^{d \times d'}$
$M[:, j]$	$\in \mathbb{R}^d$	columna j de la matriu $M \in \mathbb{R}^{d \times d'}$
$\mathbf{e}$	$\mathbb{R}^{d_e}$	representació vectorial / representació vectorial (embedding) d'un token
$\mathbf{X}$	$\mathbb{R}^{d_e \times \ell_{\mathcal{X}}}$	codificació de la seqüència de tokens primària
$\mathbf{Z}$	$\mathbb{R}^{d_e \times \ell_{\mathcal{Z}}}$	codificació de la seqüència de tokens de context
Mask	$\mathbb{R}^{\ell_{\mathcal{Z}} \times \ell_{\mathcal{X}}}$	matriu d'emascarat, determina el context d'atenció per a cada <i>token</i>
L, L_{enc}, L_{dec}	$\mathbb{N}$	nombre de capes de xarxa (codificació i descodificació)
l	$\in [L]$	índex de la capa de la xarxa
H	$\mathbb{N}$	nombre de capçals d'atenció
h	$\in [H]$	índex del capçal d'atenció
N_{data}	$\in \mathbb{N}$	(i.i.d.) grandària de la mostra
n	$\in [N_{data}]$	índex de la seqüència de mostra
η	$\in (0, \infty)$	ràtio d'aprenentatge
τ	$\in (0, \infty)$	temperatura, controla la diversitat en temps d'inferència
$\mathbf{W}_e$	$\in \mathbb{R}^{d_e \times N_V}$	matriu d' <i>embedding</i> dels tokens
$\mathbf{W}_p$	$\in \mathbb{R}^{d_e \times \ell_{max}}$	matriu d' <i>embedding</i> de posició
$\mathbf{W}_u$	$\in \mathbb{R}^{N_V \times d_e}$	matriu de conversió d' <i>embedding</i> a <i>token</i>
$\mathbf{W}_q$	$\in \mathbb{R}^{d_{attn} \times d_{\mathcal{X}}}$	paràmetres de la matriu de consulta
$\mathbf{b}_q$	$\in \mathbb{R}^{d_{attn}}$	biaix de consulta
$\mathbf{W}_k$	$\in \mathbb{R}^{d_{attn} \times d_{\mathcal{Z}}}$	paràmetres de la matriu de clau
$\mathbf{b}_k$	$\in \mathbb{R}^{d_{attn}}$	biaix de clau
$\mathbf{W}_v$	$\in \mathbb{R}^{d_{out} \times d_{\mathcal{Z}}}$	paràmetres de la matriu de valor
$\mathbf{b}_v$	$\in \mathbb{R}^{d_{attn}}$	biaix de valor
$\mathbf{W}_{qkv}$		col·lecció de paràmetres d'una capa d'atenció
$\mathbf{W}_o$	$\in \mathbb{R}^{d_{out} \times Hd_{mid}}$	paràmetres de la matriu de sortida
$\mathbf{b}_o$	$\in \mathbb{R}^{d_{out}}$	biaix de sortida
$\mathbf{W}$		col·lecció de paràmetres d'una capa de multiatenció
$\mathbf{W}_{mlp}$	$\in \mathbb{R}^{d_1 \times d_2}$	paràmetres d'una MLP del transformador
$\mathbf{b}_{mlp}$	$\in \mathbb{R}^{d_1}$	biaix corresponent a una MLP del transformador
γ	$\in \mathbb{R}^{d_e}$	paràmetre d'escalat de la capa de normalització
β	$\in \mathbb{R}^{d_e}$	paràmetre d'offset de la capa de normalització
θ	$\in \mathbb{R}^d$	col·lecció de tots els paràmetres del transformador

Bibliografia

«Top 1 Accuracy in ImageNet Image Classification». [Data de consulta: 21 de setembre de 2022]. Disponible a: <<https://paperswithcode.com/sota/image-classification-on-imagenet>>.

Akbari, H.; Yuan, L.; Qian, R.; Chuang, W.-H.; Chang, S.-F.; Cui, Y.; Gong, B. (2021). «Vatt: Transformers for multimodal self-supervised learning from raw video, audio and text». *Advances in Neural Information Processing Systems* (vol. 34, pàg. 24206–24221).

Arnab, A.; Dehghani, M.; Heigold, G.; Sun, C.; Lučić, M.; Schmid, C. (2021). «Vivit: A video vision transformer». A: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (pàg. 6836–6846).

Baevski, A.; Zhou, Y.; Mohamed, A.; Auli, M. (2020). «wav2vec 2.0: A framework for self-supervised learning of speech representations». *Advances in Neural Information Processing Systems* (vol. 33, pàg. 12449–12460).

Bahdanau, D.; Cho, K.; Bengio, Y. (2014). «Neural machine translation by jointly learning to align and translate». *arXiv preprint arXiv:1409.0473*.

Bracewell, R. N.; Bracewell, R. N. (1986). *The Fourier transform and its applications* (vol. 31999). Nova York: McGraw-Hill.

Bronstein, M. M.; Bruna, J.; Cohen, T.; Velicković, P. (2021). «Geometric deep learning: Grids, groups, graphs, geodesics, and gauges». *arXiv preprint arXiv:2104.13478*.

Carion, N.; Massa, F.; Synnaeve, G.; Usunier, N.; Kirillov, A.; Zagoruyko, S. (2020). «End-to-end object detection with transformers». A: «European conference on computer vision» (pàg. 213–229). Springer.

Chen, H.; Wang, Y.; Guo, T.; Xu, C.; Deng, Y.; Liu, Z.; Ma, S.; Xu, C.; Xu, C.; Gao, W. (2021a). «Pre-trained image processing transformer». A: «Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition» (pàg. 12299–12310).

Chen, T.; Saxena, S.; Li, L.; Fleet, D. J.; Hinton, G. (2021b). «Pix2seq: A language modeling framework for object detection». *arXiv preprint arXiv:2109.10852*.

Cheng, B.; Misra, I.; Schwing, A. G.; Kirillov, A.; Girdhar, R. (2022). «Masked-attention mask transformer for universal image segmentation». A: «Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition» (pàg. 1290–1299).

Cheng, B.; Schwing, A.; Kirillov, A. (2021). «Per-pixel classification is not all you need for semantic segmentation». *Advances in Neural Information Processing Systems* (vol. 34, pàg. 17864–17875).

Dai, X.; Chen, Y.; Yang, J.; Zhang, P.; Yuan, L.; Zhang, L. (2021). «Dynamic detr: End-to-end object detection with dynamic attention». A: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (pàg. 2988–2997).

Devlin, J.; Chang, M.-W.; Lee, K.; Toutanova, K. (2018). «Bert: Pre-training of deep bidirectional transformers for language understanding». *arXiv preprint arXiv:1810.04805*.

Ding, M.; Xiao, B.; Codella, N.; Luo, P.; Wang, J.; Yuan, L. (2022). «DaViT: Dual Attention Vision Transformers». *arXiv preprint arXiv:2204.03645*.

Dosovitskiy, A.; Beyer, L.; Kolesnikov, A.; Weissenborn, D.; Zhai, X.; Unterthiner, T.; Dehghani, M.; Minderer, M.; Heigold, G.; Gelly, S. i altres (2020). «An image is worth 16x16 words: Transformers for image recognition at scale». *arXiv preprint arXiv:2010.11929*.

Gage, P. (1994). «A new algorithm for data compression». *C Users Journal* (vol. 12, núm. 2, pàg. 23–38).

Gao, P.; Zheng, M.; Wang, X.; Dai, J.; Li, H. (2021). «Fast convergence of DETR with spatially modulated co-attention». A: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (pàg. 3621–3630).

Gavrilyuk, K.; Sanford, R.; Javan, M.; Snoek, C. G. (2020). «Actor-transformers for group activity recognition». A: «Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition» (pàg. 839–848).

Guo, D.; Ren, S.; Lu, S.; Feng, Z.; Tang, D.; Liu, S.; Zhou, L.; Duan, N.; Svyatkovskiy, A.; Fu, S. i altres (2020). «Graphcodebert: Pre-training code representations with data flow». *arXiv preprint arXiv:2009.08366*.

Hasan, M. K.; Lee, S.; Rahman, W.; Zadeh, A.; Mihalcea, R.; Morency, L.-P.; Hoque, E. (2021). «Humor knowledge enriched transformer for understanding multimodal humor». A: «Proceedings of the AAAI Conference on Artificial Intelligence» (vol. 35, pàg. 12972–12980).

He, K.; Gkioxari, G.; Dollár, P.; Girshick, R. (2017). «Mask R-CNN». A: «Proceedings of the IEEE international conference on computer vision» (pàg. 2961–2969).

He, K.; Zhang, X.; Ren, S.; Sun, J. (2016). «Deep residual learning for image recognition». A: «Proceedings of the IEEE conference on computer vision and pattern recognition» (pàg. 770–778).

Hinton, G.; Vinyals, O.; Dean, J. i altres (2015). «Distilling the knowledge in a neural network». *arXiv preprint arXiv:1503.02531* (vol. 2, núm. 7).

Hochreiter, S.; Schmidhuber, J. (1997). «Long short-term memory». *Neural computation* (vol. 9, núm. 8, pàg. 1735–1780).

Hornik, K.; Stinchcombe, M.; White, H. (1989). «Multilayer feedforward networks are universal approximators». *Neural networks* (vol. 2, núm. 5, pàg. 359–366).

Howard, J.; Ruder, S. (2018). «Universal language model fine-tuning for text classification». *arXiv preprint arXiv:1801.06146*.

Ioannou, Y. A. (2018). *Structural priors in deep neural networks*. Tesis Doctoral, University of Cambridge.

Joulin, A.; Grave, E.; Bojanowski, P.; Douze, M.; Jégou, H.; Mikolov, T. (2016). «FastText.zip: Compressing text classification models». *arXiv preprint arXiv:1612.03651*.

Kiela, D.; Bhooshan, S.; Firooz, H.; Perez, E.; Testuggine, D. (2019). «Supervised multimodal bitransformers for classifying images and text». *arXiv preprint arXiv:1909.02950*.

Kudo, T. (2018). «Subword regularization: Improving neural network translation models with multiple subword candidates». *arXiv preprint arXiv:1804.10959*.

Kumar, M.; Weissenborn, D.; Kalchbrenner, N. (2021). «Colorization transformer». *arXiv preprint arXiv:2102.04432*.

Li, R.; Yang, S.; Ross, D. A.; Kanazawa, A. (2021). «Ai choreographer: Music conditioned 3D dance generation with AIST++». A: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (pàg. 13401–13412).

Lin, J.; Yang, A.; Zhang, Y.; Liu, J.; Zhou, J.; Yang, H. (2020). «Interbert: Vision-and-language interaction for multi-modal pretraining». *arXiv preprint arXiv:2003.13198*.

Lin, T.-Y.; Dollár, P.; Girshick, R.; He, K.; Hariharan, B.; Belongie, S. (2017a). «Feature pyramid networks for object detection». A: «Proceedings of the IEEE conference on computer vision and pattern recognition» (pàg. 2117–2125).

Lin, T.-Y.; Goyal, P.; Girshick, R.; He, K.; Dollár, P. (2017b). «Focal loss for dense object detection». A: «Proceedings of the IEEE international conference on computer vision» (pàg. 2980–2988).

Lin, Y.-B.; Tseng, H.-Y.; Lee, H.-Y.; Lin, Y.-Y.; Yang, M.-H. (2021). «Exploring cross-video and cross-modality signals for weakly-supervised audio-visual video parsing». *Advances in Neural Information Processing Systems* (vol. 34, pàg. 11449–11461).

Liu, S.; Li, F.; Zhang, H.; Yang, X.; Qi, X.; Su, H.; Zhu, J.; Zhang, L. (2022). «DAB-DETR: Dynamic anchor boxes are better queries for DETR». *arXiv preprint arXiv:2201.12329*.

Liu, Y.; Ott, M.; Goyal, N.; Du, J.; Joshi, M.; Chen, D.; Levy, O.; Lewis, M.; Zettlemoyer, L.; Stoyanov, V. (2019). «Roberta: A robustly optimized bert pretraining approach». *arXiv preprint arXiv:1907.11692*.

Liu, Y.; Zhang, Y.; Wang, Y.; Hou, F.; Yuan, J.; Tian, J.; Zhang, Y.; Shi, Z.; Fan, J.; He, Z. (2021). «A survey of visual transformers». *arXiv preprint arXiv:2111.06091*.

- Lu, J.; Batra, D.; Parikh, D.; Lee, S.** (2019). «Vilbert: Pretraining task-agnostic visiolinguistic representations for vision-and-language tasks». *Advances in neural information processing systems* (vol. 32).
- Meng, D.; Chen, X.; Fan, Z.; Zeng, G.; Li, H.; Yuan, Y.; Sun, L.; Wang, J.** (2021). «Conditional detr for fast training convergence». A: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (pàg. 3651–3660).
- Mikolov, T.; Sutskever, I.; Chen, K.; Corrado, G. S.; Dean, J.** (2013). «Distributed Representations of Words and Phrases and their Compositionality». A: C. Burges; L. Bottou; M. Welling; Z. Ghahramani; K. Weinberger; Curran Associates, Inc. (eds.) «Advances in Neural Information Processing Systems» (vol. 26). Disponible a: <<https://proceedings.neurips.cc/paper/2013/file/9aa42b31882ec039965f3c4923ce901b-Paper.pdf>>.
- Nagrani, A.; Yang, S.; Arnab, A.; Jansen, A.; Schmid, C.; Sun, C.** (2021). «Attention bottlenecks for multimodal fusion». *Advances in Neural Information Processing Systems* (vol. 34, pàg. 14200–14213).
- Nussbaumer, H. J.** (1981). «The Fast Fourier Transform». A: «Fast Fourier Transform and Convolution Algorithms» (pàg. 80–111). Springer.
- Pennington, J.; Socher, R.; Manning, C. D.** (2014). «Glove: Global vectors for word representation». A: «Proceedings of the 2014 conference on empirical methods in natural language processing (EMNLP)» (pàg. 1532–1543).
- Pham, H.; Dai, Z.; Xie, Q.; Le, Q. V.** (2021). «Meta pseudo labels». A: «Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition» (pàg. 11557–11568).
- Phuong, M.; Hutter, M.** (2022). «Formal Algorithms for Transformers». *arXiv preprint arXiv:2207.09238*.
- Picek, L.; Šulc, M.; Matas, J.; Heilmann-Clausen, J.; Jeppesen, T. S.; Lind, E.** (2022). «Automatic fungi recognition: Deep learning meets mycology». *Sensors* (vol. 22, núm. 2, pàg. 633).
- Press, O.; Wolf, L.** (2016). «Using the output embedding to improve language models». *arXiv preprint arXiv:1608.05859*.
- Radford, A.; Kim, J. W.; Xu, T.; Brockman, G.; McLeavey, C.; Sutskever, I.** (2022). «Robust Speech Recognition via Large-Scale Weak Supervision».
- Radford, A.; Narasimhan, K.; Salimans, T.; Sutskever, I.** (2018). «Improving language understanding with unsupervised learning».
- Radford, A.; Wu, J.; Child, R.; Luan, D.; Amodei, D.; Sutskever, I. y otros** (2019). «Language models are unsupervised multitask learners». *OpenAI blog* (vol. 1, núm. 8, pàg. 9).
- Raffel, C.; Shazeer, N.; Roberts, A.; Lee, K.; Narang, S.; Matena, M.; Zhou, Y.; Li, W.; Liu, P. J. i altres** (2020). «Exploring the limits of transfer learning with a unified text-to-text transformer». *J. Mach. Learn. Res.* (vol. 21, núm. 140, pàg. 1–67).
- Rasmy, L.; Xiang, Y.; Xie, Z.; Tao, C.; Zhi, D.** (2021). «Med-BERT: pretrained contextualized embeddings on large-scale structured electronic health records for disease prediction». *NPJ digital medicine* (vol. 4, núm. 1, pàg. 1–13).
- Reed, S.; Zolna, K.; Parisotto, E.; Colmenarejo, S. G.; Novikov, A.; Barth-Maron, G.; Gimenez, M.; Sulsky, Y.; Kay, J.; Springenberg, J. T. y otros** (2022). «A generalist agent». *arXiv preprint arXiv:2205.06175*.
- Ronneberger, O.; Fischer, P.; Brox, T.** (2015). «U-net: Convolutional networks for biomedical image segmentation». A: «International Conference on Medical image computing and computer-assisted intervention» (pàg. 234–241). Springer.
- Shi, B.; Hsu, W.-N.; Lakhota, K.; Mohamed, A.** (2022). «Learning audio-visual speech representation by masked multimodal cluster prediction». *arXiv preprint arXiv:2201.02184*.
- Stevens, S. S.; Volkman, J.; Newman, E. B.** (1937). «A scale for the measurement of the psychological magnitude pitch». *The Journal of the Acoustical Society of America* (vol. 8, núm. 3, pàg. 185–190).
- Stewart, R.; Andriluka, M.; Ng, A. Y.** (2016). «End-to-end people detection in crowded scenes». A: «Proceedings of the IEEE conference on computer vision and pattern recognition» (pàg. 2325–2333).

- Subakan, C.; Ravanelli, M.; Cornell, S.; Bronzi, M.; Zhong, J.** (2021). «Attention is all you need in speech separation». A: «ICASSP 2021-2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)» (pàg. 21–25). IEEE.
- Sun, C.; Myers, A.; Vondrick, C.; Murphy, K.; Schmid, C.** (2019). «Videobert: A joint model for video and language representation learning». A: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (pàg. 7464–7473).
- Touvron, H.; Cord, M.; Douze, M.; Massa, F.; Sablayrolles, A.; Jégou, H.** (2021). «Training data-efficient image transformers & distillation through attention». A: «International Conference on Machine Learning» (pàg. 10.347–10.357). PMLR.
- Tsai, Y.-H. H.; Bai, S.; Liang, P. P.; Kolter, J. Z.; Morency, L.-P.; Salakhutdinov, R.** (2019). «Multimodal transformer for unaligned multimodal language sequences». A: «Proceedings of the conference. Association for Computational Linguistics. Meeting» (vol. 2019, pàg. 6558). NIH Public Access.
- Tuli, S.; Dasgupta, I.; Grant, E.; Griffiths, T. L.** (2021). «Are Convolutional Neural Networks or Transformers more like human vision?» *arXiv preprint arXiv:2105.07197*.
- Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, Ł.; Polosukhin, I.** (2017). «Attention is all you need». *Advances in neural information processing systems* (vol. 30).
- Wang, W.; Xie, E.; Li, X.; Fan, D.-P.; Song, K.; Liang, D.; Lu, T.; Luo, P.; Shao, L.** (2021). «Pyramid vision transformer: A versatile backbone for dense prediction without convolutions». A: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (pàg. 568–578).
- Wang, Y.; Zhang, X.; Yang, T.; Sun, J.** (2022). «Anchor detr: Query design for transformer-based detector». A: «Proceedings of the AAAI Conference on Artificial Intelligence» (vol. 36, pàg. 2567–2575).
- Weng, Z.; Yang, X.; Li, A.; Wu, Z.; Jiang, Y.-G.** (2021). «Semi-supervised vision transformers». *arXiv preprint arXiv:2111.11067*.
- Wu, B.; Xu, C.; Dai, X.; Wan, A.; Zhang, P.; Yan, Z.; Tomizuka, M.; Gonzalez, J.; Keutzer, K.; Vajda, P.** (2020). «Visual transformers: Token-based image representation and processing for computer vision». *arXiv preprint arXiv:2006.03677*.
- Xu, P.; Zhu, X.** (2021). «Deepchange: A large long-term person re-identification benchmark with clothes change». *arXiv e-prints* (pàg. arXiv–2105).
- Xu, P.; Zhu, X.; Clifton, D. A.** (2022). «Multimodal Learning with Transformers: A Survey». *arXiv preprint arXiv:2206.06488*.
- Yao, Z.; Ai, J.; Li, B.; Zhang, C.** (2021). «Efficient DETR: improving end-to-end object detector with dense prior». *arXiv preprint arXiv:2104.01318*.
- Yuan, L.; Chen, Y.; Wang, T.; Yu, W.; Shi, Y.; Jiang, Z.-H.; Tay, F. E.; Feng, J.; Yan, S.** (2021). «Tokens-to-token ViT: Training vision transformers from scratch on imagenet». A: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (pàg. 558–567).
- Yun, H.; Yu, Y.; Yang, W.; Lee, K.; Kim, G.** (2021). «Pano-AVQA: Grounded audio-visual question answering on 360deg videos». A: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (pàg. 2031–2041).
- Zhan, X.; Wu, Y.; Dong, X.; Wei, Y.; Lu, M.; Zhang, Y.; Xu, H.; Liang, X.** (2021). «Product1M: Towards weakly supervised instance-level product retrieval via cross-modal pretraining». A: «Proceedings of the IEEE/CVF International Conference on Computer Vision» (pàg. 11782–11791).
- Zheng, R.; Chen, J.; Ma, M.; Huang, L.** (2021a). «Fused acoustic and text encoding for multimodal bilingual pretraining and speech translation». A: «International Conference on Machine Learning» (pàg. 12736–12746). PMLR.
- Zheng, S.; Lu, J.; Zhao, H.; Zhu, X.; Luo, Z.; Wang, Y.; Fu, Y.; Feng, J.; Xiang, T.; Torr, P. H. i altres** (2021b). «Rethinking semantic segmentation from a sequence-to-sequence perspective with transformers». A: «Proceedings of the IEEE/CVF conference on computer vision and pattern recognition» (pàg. 6881–6890).
- Zhu, X.; Su, W.; Lu, L.; Li, B.; Wang, X.; Dai, J.** (2020). «Deformable DETR: Deformable transformers for end-to-end object detection». *arXiv preprint arXiv:2010.04159*.